

CHAPTER

5

CONTEXT-FREE LANGUAGES

CHAPTER SUMMARY

Having discovered the limitations of regular languages, now we go on to study more complicated languages by defining a new type of grammar, context-free grammars, and associated context-free languages. While context-free grammars are still fairly simple, they can deal with some of the languages that we know are not regular. This tells us that the family of regular languages is a proper subset of the family of context-free languages.

Two important concepts encountered here are the membership question and parsing: The membership problem (given a grammar G and a string w , find if $w \in L(G)$) is much more complicated here than it is for regular languages. So is parsing, which involves not only membership, but also finding the specific derivation that leads to w . These issues are only briefly considered in this chapter but will be studied in much greater depth in later chapters. Here we look only at brute force parsing. Brute force parsing is very general; that is, it works regardless of the form of the grammar, but it is so inefficient that it is rarely used. Its importance is primarily theoretical, as it shows that for context-free language parsing is, at least in principle, always possible. Later we will study more efficient membership and parsing algorithms.

In the last chapter, we discovered that not all languages are regular. While regular languages are effective in describing certain simple patterns, one does not need to look very far for examples of nonregular languages. The relevance of these limitations to programming languages becomes evident if we reinterpret some of the examples. If in $L = \{a^n b^n : n \geq 0\}$ we substitute a left parenthesis for a and a right parenthesis for b , then parentheses strings such as $()$ and $((()))$ are in L , but $(()$ is not. The language therefore describes a simple kind of nested structure found in programming languages, indicating that some properties of programming languages require something beyond regular languages. In order to cover this and other more complicated features we must enlarge the family of languages. This leads us to consider **context-free** languages and grammars.

We begin this chapter by defining context-free grammars and languages, illustrating the definitions with some simple examples. Next, we consider the important membership problem; in particular, we ask how we can tell if a given string is derivable from a given grammar. Explaining a sentence through its grammatical derivation is familiar to most of us from a study of natural languages and is called **parsing**. Parsing is a way of describing sentence structure. It is important whenever we need to understand the meaning of a sentence, as we do for instance in translating from one language to another. In computer science, this is relevant in interpreters, compilers, and other translating programs.

The topic of context-free languages is perhaps the most important aspect of formal language theory as it applies to programming languages. Actual programming languages have many features that can be described elegantly by means of context-free languages. What formal language theory tells us about context-free languages has important applications in the design of programming languages as well as in the construction of efficient compilers. We touch upon this briefly in Section 5.3.

5.1 CONTEXT-FREE GRAMMARS

The productions in a regular grammar are restricted in two ways: The left side must be a single variable, while the right side has a special form. To create grammars that are more powerful, we must relax some of these restrictions. By retaining the restriction on the left side, but permitting anything on the right, we get context-free grammars.

DEFINITION 5.1

A grammar $G = (V, T, S, P)$ is said to be **context-free** if all productions in P have the form

$$A \rightarrow x,$$

where $A \in V$ and $x \in (V \cup T)^*$.

A language L is said to be context-free if and only if there is a context-free grammar G such that $L = L(G)$.

Every regular grammar is context-free, so a regular language is also a context-free one. But, as we know from simple examples such as $\{a^n b^n\}$, there are nonregular languages. We have already shown in Example 1.11 that this language can be generated by a context-free grammar, so we see that the family of regular languages is a proper subset of the family of context-free languages.

Context-free grammars derive their name from the fact that the substitution of the variable on the left of a production can be made any time such a variable appears in a sentential form. It does not depend on the symbols in the rest of the sentential form (the context). This feature is the consequence of allowing only a single variable on the left side of the production.

Examples of Context-Free Languages

EXAMPLE 5.1

The grammar $G = (\{S\}, \{a, b\}, S, P)$, with productions

$$\begin{aligned} S &\rightarrow aSa, \\ S &\rightarrow bSb, \\ S &\rightarrow \lambda, \end{aligned}$$

is context-free. A typical derivation in this grammar is

$$S \Rightarrow aSa \Rightarrow aaSaa \Rightarrow aabSbaa \Rightarrow aabbbaa.$$

This, and similar derivations, make it clear that

$$L(G) = \{ww^R : w \in \{a, b\}^*\}.$$

The language is context-free, but as shown in Example 4.8, it is not regular.

EXAMPLE 5.2

The grammar G , with productions

$$\begin{aligned} S &\rightarrow abB, \\ A &\rightarrow aaBb, \\ B &\rightarrow bbAa, \\ A &\rightarrow \lambda, \end{aligned}$$

is context-free. We leave it to the reader to show that

$$L(G) = \{ab(bbaa)^n bba(ba)^n : n \geq 0\}.$$

Both of the above examples involve grammars that are not only context-free, but linear. Regular and linear grammars are clearly context-free, but a context-free grammar is not necessarily linear.

EXAMPLE 5.3

The language

$$L = \{a^n b^m : n \neq m\}$$

is context-free.

To show this, we need to produce a context-free grammar for the language. The case of $n = m$ is solved in Example 1.11, and we can build on that solution. Take the case $n > m$. We first generate a string with an equal number of a 's and b 's, then add extra a 's on the left. This is done with

$$\begin{aligned} S &\rightarrow AS_1, \\ S_1 &\rightarrow aS_1b|\lambda, \\ A &\rightarrow aA|a. \end{aligned}$$

We can use similar reasoning for the case $n < m$, and we get the answer

$$\begin{aligned} S &\rightarrow AS_1|S_1B, \\ S_1 &\rightarrow aS_1b|\lambda, \\ A &\rightarrow aA|a, \\ B &\rightarrow bB|b. \end{aligned}$$

The resulting grammar is context-free; hence, L is a context-free language. However, the grammar is not linear.

The particular form of the grammar given here was chosen for the purpose of illustration; there are many other equivalent context-free grammars. In fact, there are some simple linear ones for this language. In Exercise 28 at the end of this section you are asked to find one of them.

EXAMPLE 5.4

Consider the grammar with productions

$$S \rightarrow aSb | SS | \lambda.$$

This is another grammar that is context-free, but not linear. Some strings in $L(G)$ are $abaabb$, $aababb$, and $ababab$. It is not difficult to conjecture and prove that

$$L = \{w \in \{a, b\}^* : n_a(w) = n_b(w) \text{ and } n_a(v) \geq n_b(v), \text{ where } v \text{ is any prefix of } w\}. \quad (5.1)$$

We can see the connection with programming languages clearly if we replace a and b with left and right parentheses, respectively. The language L includes such strings as $(())$ and $()()()$ and is in fact the set of all properly nested parenthesis structures for the common programming languages.

Here again there are many other equivalent grammars. But, in contrast to Example 5.3, it is not so easy to see if there are any linear ones. We will have to wait until Chapter 8 before we can answer this question.

Leftmost and Rightmost Derivations

In a grammar that is not linear, a derivation may involve sentential forms with more than one variable. In such cases, we have a choice in the order in which variables are replaced. Take, for example, the grammar $G = (\{A, B, S\}, \{a, b\}, S, P)$ with productions

1. $S \rightarrow AB$.
2. $A \rightarrow aaA$.
3. $A \rightarrow \lambda$.
4. $B \rightarrow Bb$.
5. $B \rightarrow \lambda$.

This grammar generates the language $L(G) = \{a^{2n}b^m : n \geq 0, m \geq 0\}$. Carry out a few derivations to convince yourself of this.

Consider now the two derivations

$$S \xRightarrow{1} AB \xRightarrow{2} aaAB \xRightarrow{3} aaB \xRightarrow{4} aaBb \xRightarrow{5} aab$$

and

$$S \xRightarrow{1} AB \xRightarrow{4} ABb \xRightarrow{2} aaABb \xRightarrow{5} aaAb \xRightarrow{3} aab.$$

In order to show which production is applied, we have numbered the productions and written the appropriate number on the $\Rightarrow$ symbol. From this we see that the two derivations not only yield the same sentence but also use exactly the same productions. The difference is entirely in the order in which the productions are applied. To remove such irrelevant factors, we often require that the variables be replaced in a specific order.

DEFINITION 5.2

A derivation is said to be **leftmost** if in each step the leftmost variable in the sentential form is replaced. If in each step the rightmost variable is replaced, we call the derivation **rightmost**.

EXAMPLE 5.5

Consider the grammar with productions

$$\begin{aligned} S &\rightarrow aAB, \\ A &\rightarrow bBb, \\ B &\rightarrow A|\lambda. \end{aligned}$$

Then

$$S \Rightarrow aAB \Rightarrow abBbB \Rightarrow abAbB \Rightarrow abbBbbB \Rightarrow abbbbB \Rightarrow abbbb$$

is a leftmost derivation of the string $abbbb$. A rightmost derivation of the same string is

$$S \Rightarrow aAB \Rightarrow aA \Rightarrow abBb \Rightarrow abAb \Rightarrow abbBbb \Rightarrow abbbb.$$

Derivation Trees

A second way of showing derivations, independent of the order in which productions are used, is by a **derivation** or **parse tree**. A derivation tree is

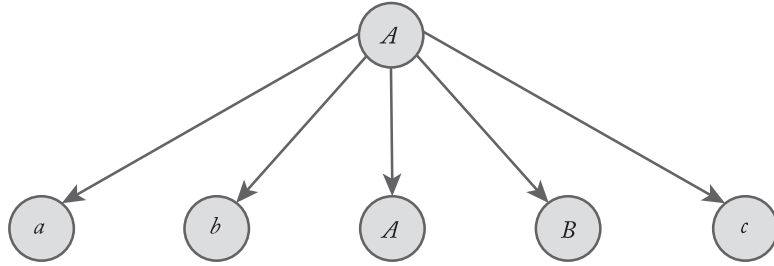


FIGURE 5.1

an ordered tree in which nodes are labeled with the left sides of productions and in which the children of a node represent its corresponding right sides. For example, Figure 5.1 shows part of a derivation tree representing the production

$$A \rightarrow abABc.$$

In a derivation tree, a node labeled with a variable occurring on the left side of a production has children consisting of the symbols on the right side of that production. Beginning with the root, labeled with the start symbol and ending in leaves that are terminals, a derivation tree shows how each variable is replaced in the derivation. The following definition makes this notion precise.

DEFINITION 5.3

Let $G = (V, T, S, P)$ be a context-free grammar. An ordered tree is a derivation tree for G if and only if it has the following properties.

1. The root is labeled S .
2. Every leaf has a label from $T \cup \{\lambda\}$.
3. Every interior vertex (a vertex that is not a leaf) has a label from V .
4. If a vertex has label $A \in V$, and its children are labeled (from left to right) $a_1, a_2, \dots, a_n$, then P must contain a production of the form

$$A \rightarrow a_1 a_2 \cdots a_n.$$

5. A leaf labeled λ has no siblings; that is, a vertex with a child labeled λ can have no other children.

A tree that has properties 3, 4, and 5, but in which 1 does not necessarily hold and in which property 2 is replaced by

- 2a. Every leaf has a label from $V \cup T \cup \{\lambda\}$,

is said to be a **partial derivation tree**.

The string of symbols obtained by reading the leaves of the tree from left to right, omitting any λ 's encountered, is said to be the **yield** of the tree. The descriptive term *left to right* can be given a precise meaning. The yield is the string of terminals in the order they are encountered when the tree is traversed in a depth-first manner, always taking the leftmost unexplored branch.

EXAMPLE 5.6

Consider the grammar G , with productions

$$S \rightarrow aAB,$$

$$A \rightarrow bBb,$$

$$B \rightarrow A|\lambda.$$

The tree in Figure 5.2 is a partial derivation tree for G , while the tree in Figure 5.3 is a derivation tree. The string $abBbB$, which is the yield of the first tree, is a sentential form of G . The yield of the second tree, $abbbb$, is a sentence of $L(G)$.

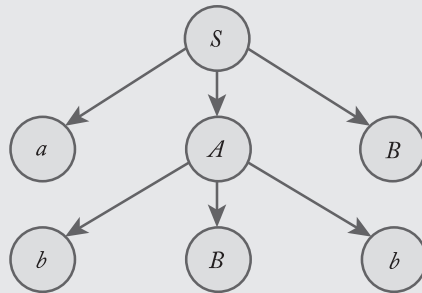


FIGURE 5.2

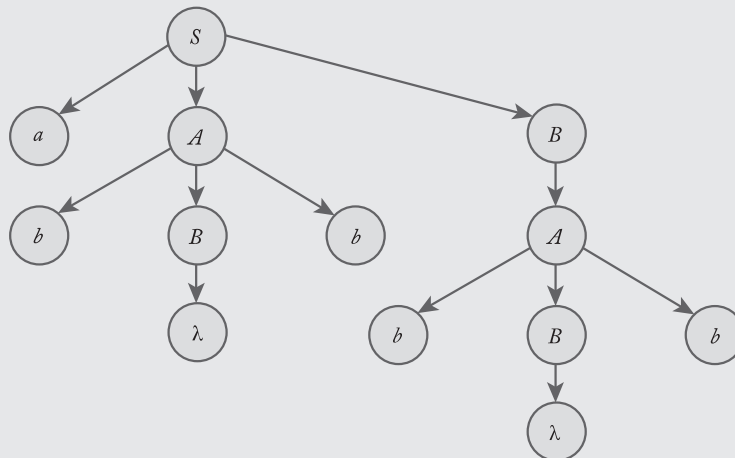


FIGURE 5.3

Relation Between Sentential Forms and Derivation Trees

Derivation trees give a very explicit and easily comprehended description of a derivation. Like transition graphs for finite automata, this explicitness is a great help in making arguments. First, though, we must establish the connection between derivations and derivation trees.

THEOREM 5.1

Let $G = (V, T, S, P)$ be a context-free grammar. Then for every $w \in L(G)$, there exists a derivation tree of G whose yield is w . Conversely, the yield of any derivation tree is in $L(G)$. Also, if t_G is any partial derivation tree for G whose root is labeled S , then the yield of t_G is a sentential form of G .

Proof: First we show that for every sentential form of $L(G)$ there is a corresponding partial derivation tree. We do this by induction on the number of steps in the derivation. As a basis, we note that the claimed result is true for every sentential form derivable in one step. Since $S \Rightarrow u$ implies that there is a production $S \rightarrow u$, this follows immediately from Definition 5.3.

Assume that for every sentential form derivable in n steps, there is a corresponding partial derivation tree. Now any w derivable in $n + 1$ steps must be such that

$$S \xRightarrow{*} xAy, \quad x, y \in (V \cup T)^*, \quad A \in V,$$

in n steps, and

$$xAy \Rightarrow xa_1a_2 \cdots a_my = w, a_i \in V \cup T.$$

Since by the inductive assumption there is a partial derivation tree with yield xAy , and since the grammar must have production $A \rightarrow a_1a_2 \cdots a_m$, we see that by expanding the leaf labeled A , we get a partial derivation tree with yield $xa_1a_2 \cdots a_my = w$. By induction, we therefore claim that the result is true for all sentential forms.

In a similar vein, we can show that every partial derivation tree represents some sentential form. We will leave this as an exercise.

Since a derivation tree is also a partial derivation tree whose leaves are terminals, it follows that every sentence in $L(G)$ is the yield of some derivation tree of G and that the yield of every derivation tree is in $L(G)$.

Derivation trees show which productions are used in obtaining a sentence, but they do not give the order of their application. Derivation trees are able to represent any derivation, reflecting the fact that this order is irrelevant, an observation that allows us to close a gap in the preceding discussion. By definition, any $w \in L(G)$ has a derivation, but we have

not claimed that it also had a leftmost or rightmost derivation. However, once we have a derivation tree, we can always get a leftmost derivation by thinking of the tree as having been built in such a way that the leftmost variable in the tree was always expanded first. Filling in a few details, we are led to the not surprising result that any $w \in L(G)$ has a leftmost and a rightmost derivation (for details, see Exercise 27 at the end of this section).

INTRODUCTORY EXERCISES

1. Find three different strings generated by the grammar

$$\begin{aligned} S &\rightarrow aSSS, \\ S &\rightarrow a. \end{aligned}$$

2. Find the language generated by the grammar

$$\begin{aligned} S &\rightarrow aSaa, \\ S &\rightarrow a. \end{aligned}$$

3. With the grammar in Example 5.4, find a leftmost derivation of $aabbab$.
4. With the grammar in Example 5.4, find a rightmost derivation of $aabbab$.
5. With the grammar in Example 5.4, find a derivation tree for $aabbab$.
6. With the grammar

$$\begin{aligned} S &\rightarrow aAB, \\ A &\rightarrow aA|\lambda, \\ B &\rightarrow bB|\lambda. \end{aligned}$$

find a derivation of $w = aabb$.

7. With the grammar

$$\begin{aligned} S &\rightarrow aSAB|aa, \\ A &\rightarrow B, \\ B &\rightarrow bbB|bb. \end{aligned}$$

find a derivation tree for $w = aaabbbb$.

EXERCISES

1. Find context-free grammars for the following languages:

- (a) $L = a^n b^n$, n is even.
- (b) $L = a^n b^n$, n is odd.

- (c) $L = a^n b^n$, n is a multiple of three.
- (d) $L = a^n b^n$, n is not a multiple of three.
2. Show a derivation tree for $w = aaabbaaa$ using the grammar in Example 5.1.
 3. Draw the derivation tree corresponding to the derivation in Example 5.1.
 4. Give a derivation tree for $w = abbbaabbaba$ for the grammar in Example 5.2.
 5. Complete the arguments in Example 5.2, showing that the language given is generated by the grammar.
 6. Show that the grammar in Example 5.4 does in fact generate the language described in Equation 5.1.
 7. Is the language in Example 5.2 regular?
 8. Complete the proof in Theorem 5.1 by showing that the yield of every partial derivation tree with root S is a sentential form of G .
 9. Find context-free grammars for the following languages (with $n \geq 0$, $m \geq 0$).
 - (a) $L = \{a^n b^m : n \leq m + 3\}$.
 - (b) $L = \{a^n b^m : n = m - 1\}$.
 - (c) $L = \{a^n b^m : n \neq 2m\}$.
 - (d) $L = \{a^n b^m : 2n \leq m \leq 3n\}$.
 - (e) $L = \{w \in \{a, b\}^* : n_a(w) \neq n_b(w)\}$.
 - (f) $L = \{w \in \{a, b\}^* : n_a(v) \geq n_b(v), \text{ where } v \text{ is any prefix of } w\}$.
 - (g) $L = \{w \in \{a, b\}^* : n_a(w) = 2n_b(w) + 1\}$.
 - (h) $L = \{w \in \{a, b\}^* : n_a(w) = n_b(w) + 2\}$.
 10. What language does the grammar

$$S \rightarrow aSB|bSA$$

$$S \rightarrow \lambda$$

$$A \rightarrow a$$

$$B \rightarrow b$$

generate?

11. What language does the grammar

$$S \rightarrow aaSbb|SS$$

$$S \rightarrow \lambda$$

generate?

12. Find context-free grammars for the following languages (with $n \geq 0$, $m \geq 0$, $k \geq 0$):
 - (a) $L = \{a^n b^m c^k : n = m \text{ or } m \leq k\}$.
 - (b) $L = \{a^n b^m c^k : n = m \text{ or } m \neq k\}$.

- (c) $L = \{a^n b^m c^k, k = n + m\}.$
- (d) $L = \{a^n b^m c^k, k = |n - m|\}.$
- (e) $L = \{a^n b^m c^k, k = n + 2m\}.$
- (f) $L = \{a^n b^m c^k, k \neq n + m\}.$

13. Find a context-free grammar for

$$L = \{w \in \{a, b, c\}^* : n_a(w) + n_b(w) \neq n_c(w)\}.$$

14. Show that $L = \{w \in \{a, b, c\}^* : |w| = 3n_a(w)\}$ is a context-free language.

15. Find a context-free grammar for $\Sigma = \{a, b\}$ for the language

$$L = \left\{a^n w w^R b^n : w \in \Sigma^*, n \geq 1\right\}.$$

16. Let $L = \{a^n b^n : n \geq 0\}.$

- (a) Show that L^2 is context-free.
- (b) Show that L^k is context-free for all $k > 1.$
- (c) Show that $\overline{L}$ and L^* are context-free.

17. Let L_1 be the language in Exercise 12(a) and L_2 be the language in Exercise 12(d). Show that $L_1 \cup L_2$ is a context-free language.

18. Show that the following language is context-free:

$$L = \left\{uvwv^R : u, v, w \in \{a, b\}^+, |u| = |w| = 2\right\}.$$

- 19. Show that the complement of the language in Example 5.1 is context-free.
- 20. Show that the complement of the language in Exercise 12(c) is context-free.
- 21. Show that the language $L = \{w_1 c w_2 : w_1, w_2 \in \{a, b\}^+, w_1 \neq w_2^R\}$, with $\Sigma = \{a, b, c\}$, is context-free.
- 22. Show a derivation tree for the string $aabbbb$ with the grammar

$$\begin{aligned} S &\rightarrow AB|\lambda, \\ A &\rightarrow aB, \\ B &\rightarrow Sb. \end{aligned}$$

Give a verbal description of the language generated by this grammar.

23. Consider the grammar with productions

$$\begin{aligned} S &\rightarrow aaB, \\ A &\rightarrow bBb|\lambda, \\ B &\rightarrow Aa. \end{aligned}$$

Show that the string $aabbabba$ is not in the language generated by this grammar.

24. Define what one might mean by properly nested parenthesis structures involving two kinds of parentheses, say $()$ and $[\]$. Intuitively, properly nested strings in this situation are $([\])$, $([[\]])$, $([\])$, but not $([\])$ or $([\])$. Using your definition, give a context-free grammar for generating all properly nested parentheses.
25. Find a context-free grammar for the set of all regular expressions on the alphabet $\{a, b\}$.
26. Find a context-free grammar that can generate all the production rules for context-free grammars with $T = \{a, b\}$ and $V = \{A, B, C\}$.
27. Prove that if G is a context-free grammar, then every $w \in L(G)$ has a leftmost and rightmost derivation. Give an algorithm for finding such derivations from a derivation tree.
28. Find a linear grammar for the language in Example 5.3.
29. Let $G = (V, T, S, P)$ be a context-free grammar such that every one of its productions is of the form $A \rightarrow v$, with $|v| = k > 1$. Show that the derivation tree for any $w \in L(G)$ has a height h such that

$$\log_k |w| \leq h \leq \frac{(|w| - 1)}{k - 1}.$$

5.2 PARSING AND AMBIGUITY

We have so far concentrated on the generative aspects of grammars. Given a grammar G , we studied the set of strings that can be derived using G . In cases of practical applications, we are also concerned with the analytical side of the grammar: Given a string w of terminals, we want to know whether or not w is in $L(G)$. If so, we may want to find a derivation of w . An algorithm that can tell us whether w is in $L(G)$ is a membership algorithm. The term **parsing** describes finding a sequence of productions by which a $w \in L(G)$ is derived.

Parsing and Membership

Given a string w in $L(G)$, we can parse it in a rather obvious fashion: We systematically construct all possible (say, leftmost) derivations and see whether any of them match w . Specifically, we start at round one by looking at all productions of the form

$$S \rightarrow x,$$

finding all x that can be derived from S in one step. If none of these results in a match with w , we go to the next round, in which we apply all applicable productions to the leftmost variable of every x . This gives us a set of sentential forms, some of them possibly leading to w . On each

subsequent round, we again take all leftmost variables and apply all possible productions. It may be that some of these sentential forms can be rejected on the grounds that w can never be derived from them, but in general, we will have on each round a set of possible sentential forms. After the first round, we have sentential forms that can be derived by applying a single production, after the second round we have the sentential forms that can be derived in two steps, and so on. If $w \in L(G)$, then it must have a leftmost derivation of finite length. Thus, the method will eventually give a leftmost derivation of w .

For reference below, we will call this **exhaustive search parsing** or **brute force parsing**. It is a form of **top-down parsing**, which we can view as the construction of a derivation tree from the root down.

EXAMPLE 5.7

Consider the grammar

$$S \rightarrow SS \mid aSb \mid bSa \mid \lambda$$

and the string $w = aabb$. Round one gives us

1. $S \Rightarrow SS$,
2. $S \Rightarrow aSb$,
3. $S \Rightarrow bSa$,
4. $S \Rightarrow \lambda$.

The last two of these can be removed from further consideration for obvious reasons. Round two then yields sentential forms

$$\begin{aligned} S &\Rightarrow SS \Rightarrow SSS, \\ S &\Rightarrow SS \Rightarrow aSbS, \\ S &\Rightarrow SS \Rightarrow bSaS, \\ S &\Rightarrow SS \Rightarrow S, \end{aligned}$$

which are obtained by replacing the leftmost S in sentential form 1 with all applicable substitutes. Similarly, from sentential form 2 we get the additional sentential forms

$$\begin{aligned} S &\Rightarrow aSb \Rightarrow aSSb, \\ S &\Rightarrow aSb \Rightarrow aaSbb, \\ S &\Rightarrow aSb \Rightarrow abSab, \\ S &\Rightarrow aSb \Rightarrow ab. \end{aligned}$$

Again, several of these can be removed from contention. On the next round, we find the actual target string from the sequence

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb.$$

Therefore, $aabb$ is in the language generated by the grammar under consideration.

Exhaustive search parsing has serious flaws. The most obvious one is its tediousness; it is not to be used where efficient parsing is required. But even when efficiency is a secondary issue, there is a more pertinent objection. While the method always parses a $w \in L(G)$, it is possible that it never terminates for strings not in $L(G)$. This is certainly the case in the previous example; with $w = abb$, the method will go on producing trial sentential forms indefinitely unless we build into it some way of stopping.

The problem of nontermination of exhaustive search parsing is relatively easy to overcome if we restrict the form that the grammar can have. If we examine Example 5.7, we see that the difficulty comes from the productions $S \rightarrow \lambda$; this production can be used to decrease the length of successive sentential forms, so that we cannot tell easily when to stop. If we do not have any such productions, then we have many fewer difficulties. In fact, there are two types of productions we want to rule out, those of the form $A \rightarrow \lambda$ as well as those of the form $A \rightarrow B$. As we will see in the next chapter, this restriction does not affect the power of the resulting grammars in any significant way.

EXAMPLE 5.8

The grammar

$$S \rightarrow SS \mid aSb \mid bSa \mid ab \mid ba$$

satisfies the given requirements. It generates the language in Example 5.7 without the empty string.

Given any $w \in \{a, b\}^+$, the exhaustive search parsing method will always terminate in no more than $|w|$ rounds. This is clear because the length of the sentential form grows by at least one symbol in each round. After $|w|$ rounds we have either produced a parsing or we know that $w \notin L(G)$.

The idea in this example can be generalized and made into a theorem for context-free languages in general.

THEOREM 5.2

Suppose that $G = (V, T, S, P)$ is a context-free grammar that does not have any rules of the form

$$A \rightarrow \lambda,$$

or

$$A \rightarrow B,$$

where $A, B \in V$. Then the exhaustive search parsing method can be made into an algorithm that, for any $w \in \Sigma^*$, either produces a parsing of w or tells us that no parsing is possible.

Proof: For each sentential form, consider both its length and the number of terminal symbols. Each step in the derivation increases at least one of these. Since neither the length of a sentential form nor the number of terminal symbols can exceed $|w|$, a derivation cannot involve more than $2|w|$ rounds, at which time we either have a successful parsing or w cannot be generated by the grammar. ■

While the exhaustive search method gives a theoretical guarantee that parsing can always be done, its practical usefulness is limited because the number of sentential forms generated by it may be excessively large. Exactly how many sentential forms are generated differs from case to case; no precise general result can be established, but we can put some rough upper bounds on it. If we restrict ourselves to leftmost derivations, we can have no more than $|P|$ sentential forms after one round, no more than $|P|^2$ sentential forms after the second round, and so on. In the proof of Theorem 5.2, we observed that parsing cannot involve more than $2|w|$ rounds; therefore, the total number of sentential forms cannot exceed

$$\begin{aligned} M &= |P| + |P|^2 + \cdots + |P|^{2|w|} \\ &= O(P^{2|w|+1}). \end{aligned} \tag{5.2}$$

This indicates that the work for exhaustive search parsing may grow exponentially with the length of the string, making the cost of the method prohibitive. Of course, Equation (5.2) is only a bound, and often the number of sentential forms is much smaller. Nevertheless, practical observation shows that exhaustive search parsing is very inefficient in most cases.

The construction of more efficient parsing methods for context-free grammars is a complicated matter that belongs to a course on compilers, but we explore two such methods in Chapters 15–17. We will not pursue it here except for some isolated results.

THEOREM 5.3

For every context-free grammar there exists an algorithm that parses any $w \in L(G)$ in a number of steps proportional to $|w|^3$.

There are several known methods to achieve this, but all of them are sufficiently complicated that we cannot even describe them without developing some additional results. In Section 6.3 we will take this question up again briefly. More details can be found in Harrison 1978 and Hopcroft and Ullman 1979. One reason for not pursuing this in detail is that even these algorithms are unsatisfactory. A method in which the work rises with the third power of the length of the string, while better than an exponential algorithm, is still quite inefficient, and a parser based on it would need an excessive amount of time to analyze even a moderately long program. What we would like to have is a parsing method that takes time proportional to the length of the string. We refer to such a method as a **linear time** parsing algorithm. We do not know any linear time parsing methods for context-free languages in general, but such algorithms can be found for restricted, but important, special cases.

DEFINITION 5.4

A context-free grammar $G = (V, T, S, P)$ is said to be a **simple grammar** or **s-grammar** if all its productions are of the form

$$A \rightarrow ax,$$

where $A \in V$, $a \in T$, $x \in V^*$, and any pair (A, a) occurs at most once in P .

EXAMPLE 5.9

The grammar

$$S \rightarrow aS \mid bSS \mid c$$

is an s-grammar. The grammar

$$S \rightarrow aS \mid bSS \mid aSS \mid c$$

is not an s-grammar because the pair (S, a) occurs in the two productions $S \rightarrow aS$ and $S \rightarrow aSS$.

While s-grammars are quite restrictive, they are of some interest. As we will see in the next section, many features of common programming languages can be described by s-grammars.

If G is an s-grammar, then any string w in $L(G)$ can be parsed with an effort proportional to $|w|$. To see this, look at the exhaustive search method and the string $w = a_1 a_2 \cdots a_n$. Since there can be at most one rule with S on the left, and starting with a_1 on the right, the derivation must begin with

$$S \Rightarrow a_1 A_1 A_2 \cdots A_m.$$

Next, we substitute for the variable A_1 , but since again there is at most one choice, we must have

$$S \Rightarrow^* a_1 a_2 B_1 B_2 \cdots A_2 \cdots A_m.$$

We see from this that each step produces one terminal symbol, and hence the whole process must be completed in no more than $|w|$ steps.

Ambiguity in Grammars and Languages

On the basis of our argument we can claim that given any $w \in L(G)$, exhaustive search parsing will produce a derivation tree for w . We say “a” derivation tree rather than “the” derivation tree because of the possibility that a number of different derivation trees may exist. This situation is referred to as **ambiguity**.

DEFINITION 5.5

A context-free grammar G is said to be **ambiguous** if there exists some $w \in L(G)$ that has at least two distinct derivation trees. Alternatively, ambiguity implies the existence of two or more leftmost or rightmost derivations.

EXAMPLE 5.10

The grammar in Example 5.4, with productions $S \rightarrow aSb \mid SS \mid \lambda$, is ambiguous. The sentence $aabb$ has the two derivation trees shown in Figure 5.4.

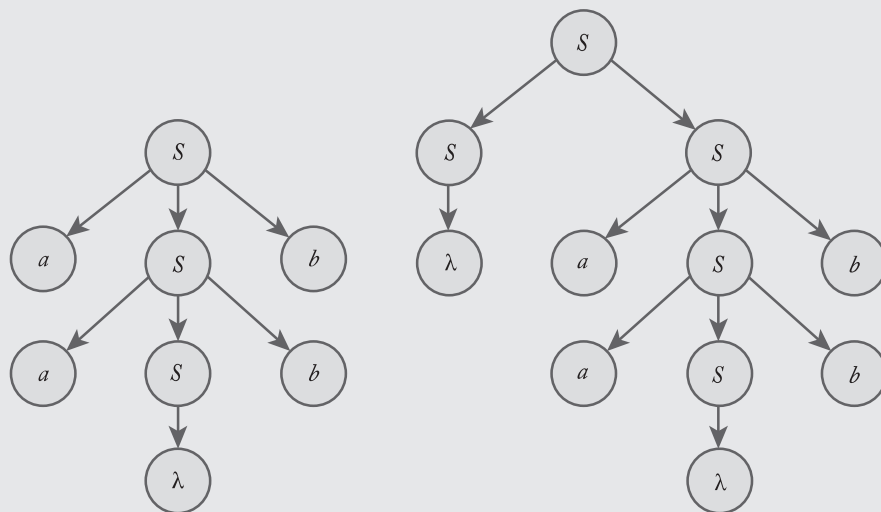


FIGURE 5.4

Ambiguity is a common feature of natural languages, where it is tolerated and dealt with in a variety of ways. In programming languages, where there should be only one interpretation of each statement, ambiguity must be removed when possible. Often we can achieve this by rewriting the grammar in an equivalent, unambiguous form.

EXAMPLE 5.11

Consider the grammar $G = (V, T, E, P)$ with

$$V = \{E, I\},$$

$$T = \{a, b, c, +, *, (,)\},$$

and productions

$$\begin{aligned} E &\rightarrow I, \\ E &\rightarrow E + E, \\ E &\rightarrow E * E, \\ E &\rightarrow (E), \\ I &\rightarrow a|b|c. \end{aligned}$$

The strings $(a + b) * c$ and $a * b + c$ are in $L(G)$. It is easy to see that this grammar generates a restricted subset of arithmetic expressions

for C-like programming languages. The grammar is ambiguous. For instance, the string $a + b * c$ has two different derivation trees, as shown in Figure 5.5.

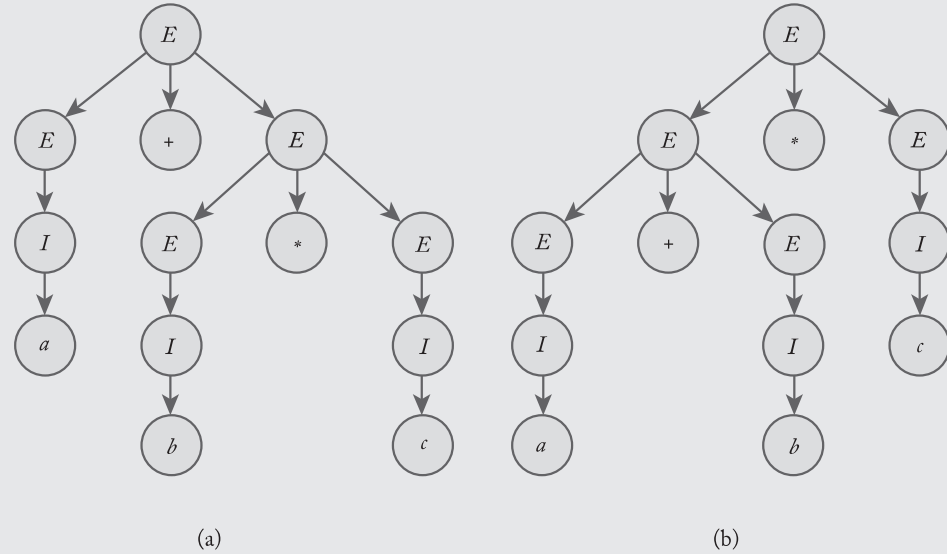


FIGURE 5.5 Two derivation trees for $a + b * c$.

One way to resolve the ambiguity is, as is done in programming manuals, to associate precedence rules with the operators $+$ and $*$. Since $*$ normally has higher precedence than $+$, we would take Figure 5.5(a) as the correct parsing because it indicates that $b * c$ is a subexpression to be evaluated before performing the addition. However, this resolution is completely outside the grammar. It is better to rewrite the grammar so that only one parsing is possible.

EXAMPLE 5.12

To rewrite the grammar in Example 5.11 we introduce new variables, taking V as $\{E, T, F, I\}$, and replacing the productions with

$$\begin{aligned} E &\rightarrow T, \\ T &\rightarrow F, \\ F &\rightarrow I, \\ E &\rightarrow E + T, \\ T &\rightarrow T * F, \end{aligned}$$

$$F \rightarrow (E),$$

$$I \rightarrow a \mid b \mid c.$$

A derivation tree of the sentence $a + b * c$ is shown in Figure 5.6. No other derivation tree is possible for this string: The grammar is unambiguous. It is also equivalent to the grammar in Example 5.11. It is not too hard to justify these claims in this specific instance, but, in general, the questions of whether a given context-free grammar is ambiguous or whether two given context-free grammars are equivalent are very difficult to answer. In fact, we will later show that there are no general algorithms by which these questions can always be resolved.

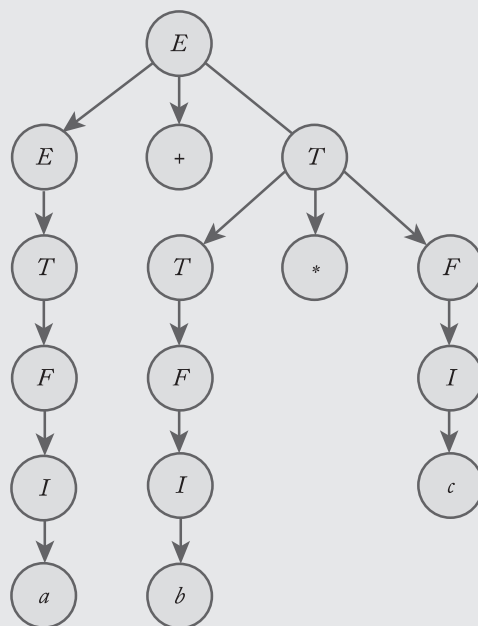


FIGURE 5.6

In the foregoing example the ambiguity came from the grammar in the sense that it could be removed by finding an equivalent unambiguous grammar. In some instances, however, this is not possible because the ambiguity is in the language.

DEFINITION 5.6

If L is a context-free language for which there exists an unambiguous grammar, then L is said to be unambiguous. If every grammar that generates L is ambiguous, then the language is called **inherently ambiguous**.

It is a somewhat difficult matter even to exhibit an inherently ambiguous language. The best we can do here is give an example with some reasonably plausible claim that it is inherently ambiguous.

EXAMPLE 5.13

The language

$$L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\},$$

with n and m nonnegative, is an inherently ambiguous context-free language.

That L is context-free is easy to show. Notice that

$$L = L_1 \cup L_2,$$

where L_1 is generated by

$$\begin{aligned} S_1 &\rightarrow S_1 c | A, \\ A &\rightarrow a A b | \lambda \end{aligned}$$

and L_2 is given by an analogous grammar with start symbol S_2 and productions

$$\begin{aligned} S_2 &\rightarrow a S_2 | B, \\ B &\rightarrow b B c | \lambda. \end{aligned}$$

Then L is generated by the combination of these two grammars with the additional production

$$S \rightarrow S_1 | S_2.$$

The grammar is ambiguous since the string $a^n b^n c^n$ has two distinct derivations, one starting with $S \Rightarrow S_1$, the other with $S \Rightarrow S_2$. It does not, of course, follow from this that L is inherently ambiguous as there might exist some other unambiguous grammars for it. But in some way L_1 and L_2 have conflicting requirements, the first putting a restriction on the number of a 's and b 's, while the second does the same for b 's and c 's. A few tries will quickly convince you of the impossibility of combining these requirements in a single set of rules that cover the case $n = m$ uniquely. A rigorous argument, though, is quite technical. One proof can be found in Harrison 1978.

INTRODUCTORY EXERCISES

1. Use brute force parsing to show that the string $abbba$ is not in the language generated by the grammar

$$S \rightarrow aSb|bSa|ab|ba.$$

2. Show that the grammar

$$S \rightarrow aSb|ab|\lambda$$

is ambiguous.

3. Even though the grammar in the preceding exercise is ambiguous, the language that it generates, namely $L = \{a^n b^n, n \geq 0\}$, is not inherently ambiguous. Show that this is so by finding an unambiguous grammar for L .
4. Verify that

$$S \rightarrow aA, A \rightarrow aAB, A \rightarrow b, B \rightarrow b$$

is an s-grammar. What language does it generate?

EXERCISES

1. Show that

$$\begin{aligned} S &\rightarrow aS|bS|cA, \\ A &\rightarrow aA|bS \end{aligned}$$

is an s-grammar.

2. Show that every s-grammar is unambiguous.
3. Find an s-grammar for $L(aaa^*b + ab^*)$.
4. Find an s-grammar for $L = \{a^n b^n : n \geq 2\}$.
5. Find an s-grammar for $L = \{a^n b^{2n} : n \geq 2\}$.
6. Find an s-grammar for $L = \{a^n b^{n+1} : n \geq 1\}$.
7. Let $G = (V, T, S, P)$ be an s-grammar. Give an expression for the maximum size of P in terms of $|V|$ and $|T|$.
8. Show that the following grammar is ambiguous:

$$\begin{aligned} S &\rightarrow AB|aaaB, \\ A &\rightarrow a|Aa, \\ B &\rightarrow b. \end{aligned}$$

9. Construct an unambiguous grammar equivalent to the grammar in Exercise 8.
10. Give the derivation tree for $((a+b)*c+d)$, using the grammar in Example 5.12.
11. Give the derivation tree for $a*b+((c+d))$, using the grammar in Example 5.12.

12. Give the derivation tree for $((a + b) * c) + a + b$, using the grammar in Example 5.12.
13. Show that a regular language cannot be inherently ambiguous.
14. Give an unambiguous grammar that generates the set of all regular expressions on $\Sigma = \{a, b\}$.
15. Is it possible for a regular grammar to be ambiguous?
16. Show that the language $L = \{ww^R : w \in \{a, b\}^*\}$ is not inherently ambiguous.
17. Show that the following grammar is ambiguous:

$$S \rightarrow aSbS \mid bSaS \mid \lambda.$$

18. Show that the grammar in Example 5.4 is ambiguous but that the language denoted by it is not.
19. Show that the grammar in Example 1.13 is ambiguous.
20. Show that the grammar in Example 5.5 is unambiguous.
21. Use the exhaustive search parsing method to parse the string $abbbbb$ with the grammar in Example 5.5. In general, how many rounds will be needed to parse any string w in this language?
22. Is the string $aabbababb$ in the language generated by the grammar $S \rightarrow aSS \mid b$?
23. Show that the grammar in Example 1.14 is unambiguous.
24. Prove the following result. Let $G = (V, T, S, P)$ be a context-free grammar in which every $A \in V$ occurs on the left side of at most one production. Then G is unambiguous.
25. Find a grammar equivalent to that in Example 5.5 that satisfies the conditions of Theorem 5.2.

5.3 CONTEXT-FREE GRAMMARS AND PROGRAMMING LANGUAGES

One of the most important uses of the theory of formal languages is in the definition of programming languages and in the construction of interpreters and compilers for them. The basic problem here is to define a programming language precisely and to use this definition as the starting point for the writing of efficient and reliable translation programs. Both regular and context-free languages are important in achieving this. As we have seen, regular languages are used in the recognition of certain simple patterns that occur in programming languages, but as we argue in the introduction to this chapter, we need context-free languages to model more complicated aspects.

As with most other languages, we can define a programming language by a grammar. It is traditional in writing on programming languages to

use a convention for specifying grammars called the *Backus-Naur* form or BNF. This form is in essence the same as the notation we have used here, but the appearance is different. In BNF, variables are enclosed in triangular brackets. Terminal symbols are written without any special marking. BNF also uses subsidiary symbols such as $|$, much in the way we have done. Thus, the grammar in Example 5.12 might appear in BNF as

$$\begin{aligned}\langle expression \rangle &::= \langle term \rangle | \langle expression \rangle + \langle term \rangle, \\ \langle term \rangle &::= \langle factor \rangle | \langle term \rangle * \langle factor \rangle,\end{aligned}$$

and so on. The symbols $+$ and $*$ are terminals. The symbol $|$ is used as an alternator as in our notation, but $::=$ is used instead of $\rightarrow$. BNF descriptions of programming languages tend to use more explicit variable identifiers to make the intent of the production explicit. But otherwise there are no significant differences between the two notations.

Many parts of C-like programming languages are susceptible to definition by restricted forms of context-free grammars. For example, the *while* statement in C can be defined as

$$\langle while_statement \rangle ::= while \langle expression \rangle \langle statement \rangle.$$

Here the keyword *while* is a terminal symbol. All other terms are variables, which still have to be defined. If we check this against Definition 5.4, we see that this looks like an s-grammar production. The variable $\langle while_statement \rangle$ on the left is always associated with the terminal *while* on the right. For this reason such a statement is easily and efficiently parsed. We see here a reason why we use keywords in programming languages. Keywords not only provide some visual structure that can guide the reader of a program, but also make the work of a compiler much easier.

Unfortunately, not all features of a typical programming language can be expressed by an s-grammar. The rules for $\langle expression \rangle$ above are not of this type, so that parsing becomes less obvious. The question then arises, “What grammatical rules we can permit and still parse efficiently?” In compilers, extensive use has been made of what are called LL and LR grammars. These grammars have the ability to express the less obvious features of a programming language, yet allow us to parse in linear time. This is not a simple matter, and much of it is beyond the scope of our discussion. We will briefly touch on this topic in Chapter 6, but for our purposes it suffices to realize that such grammars exist and have been widely studied.

In connection with this, the issue of ambiguity takes on added significance. The specification of a programming language must be unambiguous, otherwise a program may yield very different results when processed by different compilers or run on different systems. As Example 5.11 shows, a naive approach can easily introduce ambiguity in the grammar. To avoid such mistakes we must be able to recognize and remove ambiguities. A related question is whether a language is or is not inherently ambiguous.

What we need for this purpose are algorithms for detecting and removing ambiguities in context-free grammars and for deciding whether or not a context-free language is inherently ambiguous. Unfortunately, these are very difficult tasks, impossible in the most general sense, as we will see later.

Those aspects of a programming language that can be modeled by a context-free grammar are usually referred to as its **syntax**. However, it is normally the case that not all programs that are syntactically correct in this sense are in fact acceptable programs. For C, the usual BNF definition allows constructs such as

char *a, b, c*;

followed by

c = 3.2;

This combination is not acceptable to C compilers since it violates the constraint, “a character variable cannot be assigned a real value.” Context-free grammars cannot express the fact that type clashes may not be permitted. Such rules are part of programming language semantics, since they have to do with how we interpret the meaning of a particular construct.

Programming language semantics are a complicated matter. Nothing as elegant and concise as context-free grammars exists for the specification of programming language semantics, and consequently some semantic features may be poorly defined or ambiguous. It is an ongoing concern both in programming languages and in formal language theory to find effective methods for defining programming language semantics. Several methods have been proposed, but none of them has been as universally accepted and are as successful for semantic definition as context-free languages have been for syntax.

EXERCISES

1. Suppose that in a certain programming language numbers are constructed according to the following rules:
 - (a) A sign is optional.
 - (b) The value field consists of two nonempty parts, separated by a decimal point.
 - (c) An exponent field is optional. When present, the exponent field must consist of the letter **e** and a signed two-digit integer.

Find a grammar that formalizes these requirements.

- 2.** Consult a book on C for formal definitions of the following constructs:

Literal

For statement

If-else statement

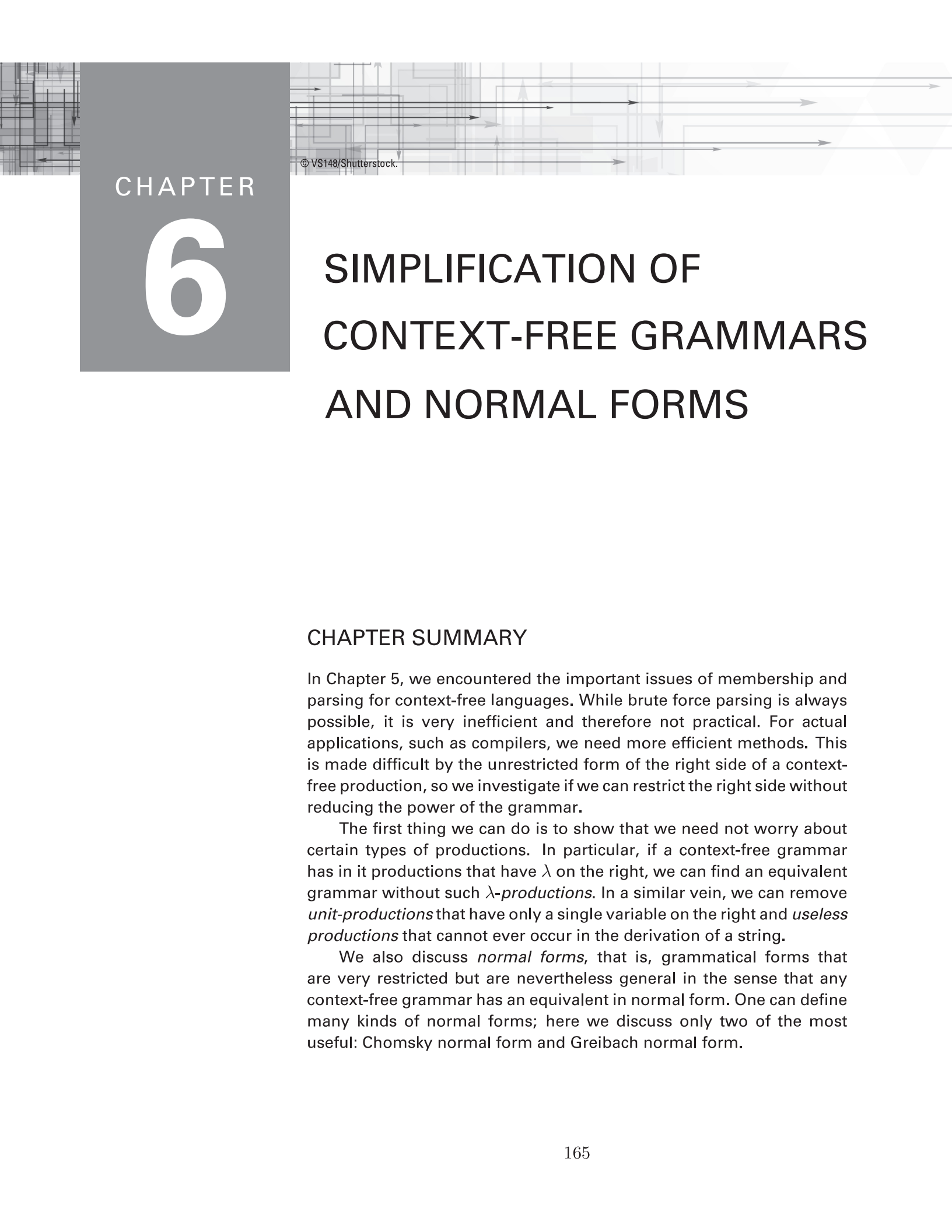
Do statement

Compound statement

Return statement

- 3.** Find examples of features of C that cannot be described by context-free grammars.





CHAPTER

6

SIMPLIFICATION OF CONTEXT-FREE GRAMMARS AND NORMAL FORMS

CHAPTER SUMMARY

In Chapter 5, we encountered the important issues of membership and parsing for context-free languages. While brute force parsing is always possible, it is very inefficient and therefore not practical. For actual applications, such as compilers, we need more efficient methods. This is made difficult by the unrestricted form of the right side of a context-free production, so we investigate if we can restrict the right side without reducing the power of the grammar.

The first thing we can do is to show that we need not worry about certain types of productions. In particular, if a context-free grammar has in it productions that have λ on the right, we can find an equivalent grammar without such λ -productions. In a similar vein, we can remove *unit-productions* that have only a single variable on the right and *useless productions* that cannot ever occur in the derivation of a string.

We also discuss *normal forms*, that is, grammatical forms that are very restricted but are nevertheless general in the sense that any context-free grammar has an equivalent in normal form. One can define many kinds of normal forms; here we discuss only two of the most useful: Chomsky normal form and Greibach normal form.

Before we can study context-free languages in greater depth, we must attend to some technical matters. The definition of a context-free grammar imposes no restriction whatsoever on the right side of a production. However, complete freedom is not necessary and, in fact, is a detriment in some arguments. In Theorem 5.2, we see the convenience of certain restrictions on grammatical forms; eliminating rules of the form $A \rightarrow \lambda$ and $A \rightarrow B$ make the arguments easier. In many instances, it is desirable to place even more stringent restrictions on the grammar. Because of this, we need to look at methods for transforming an arbitrary context-free grammar into an equivalent one that satisfies certain restrictions on its form. In this chapter we study several transformations and substitutions that will be useful in subsequent discussions.

We also investigate **normal forms** for context-free grammars. A normal form is one that, although restricted, is broad enough so that any grammar has an equivalent normal-form version. We introduce two of the most useful of these, the **Chomsky normal form** and the **Greibach normal form**. Both have many practical and theoretical uses. An immediate application of the Chomsky normal form to parsing is given in Section 6.3.

The somewhat tedious nature of the material in this chapter lies in the fact that many of the arguments are manipulative and give little intuitive insight. For our purposes, this technical aspect is relatively unimportant and can be read casually. The various conclusions are significant; they will be used many times in later discussions.

6.1 METHODS FOR TRANSFORMING GRAMMARS

We first raise an issue that is somewhat of a nuisance with grammars and languages in general: the presence of the empty string. The empty string plays a rather singular role in many theorems and proofs, and it is often necessary to give it special attention. We prefer to remove it from consideration altogether, looking only at languages that do not contain λ . In doing so, we do not lose generality, as we see from the following considerations. Let L be any context-free language, and let $G = (V, T, S, P)$ be a context-free grammar for $L - \{\lambda\}$. Then the grammar we obtain by adding to V the new variable S_0 , making S_0 the start variable, and adding to P the productions

$$S_0 \rightarrow S|\lambda$$

generates L . Therefore, any nontrivial conclusion we can make for $L - \{\lambda\}$ will almost certainly transfer to L . Also, given any context-free grammar G , there is a method for obtaining $\hat{G}$ such that $L(\hat{G}) = L(G) - \{\lambda\}$ (see Exercises 15 and 16 at the end of this section). Consequently, for all practical purposes, there is no difference between context-free languages that include λ and those that do not. For the rest of this chapter, unless otherwise stated, we will restrict our discussion to λ -free languages.

A Useful Substitution Rule

Many rules govern generating equivalent grammars by means of substitutions. Here we give one that is very useful for simplifying grammars in various ways. We will not define the term *simplification* precisely, but we will use it nevertheless. What we mean by it is the removal of certain types of undesirable productions; the process does not necessarily result in an actual reduction of the number of rules.

THEOREM 6.1

Let $G = (V, T, S, P)$ be a context-free grammar. Suppose that P contains a production of the form

$$A \rightarrow x_1 B x_2.$$

Assume that A and B are different variables and that

$$B \rightarrow y_1 | y_2 | \cdots | y_n$$

is the set of all productions in P that have B as the left side. Let $\hat{G} = (V, T, S, \hat{P})$ be the grammar in which $\hat{P}$ is constructed by deleting

$$A \rightarrow x_1 B x_2 \tag{6.1}$$

from P , and adding to it

$$A \rightarrow x_1 y_1 x_2 | x_1 y_2 x_2 | \cdots | x_1 y_n x_2.$$

Then

$$L(\hat{G}) = L(G).$$

Proof: Suppose that $w \in L(G)$, so that

$$S \xRightarrow{*}_G w.$$

The subscript on the derivation sign $\Rightarrow$ is used here to distinguish between derivations with different grammars. If this derivation does not involve the production (6.1), then obviously

$$S \xRightarrow{*}_{\hat{G}} w.$$

If it does, then look at the derivation the first time (6.1) is used. The B so introduced eventually has to be replaced; we lose nothing by assuming that this is done immediately (see Exercise 20 at the end of this section). Thus

$$S \xRightarrow{*}_G u_1 A u_2 \Rightarrow_G u_1 x_1 B x_2 u_2 \Rightarrow_G u_1 x_1 y_j x_2 u_2.$$

But with grammar $\widehat{G}$ we can get

$$S \xRightarrow{*}_{\widehat{G}} u_1 A u_2 \Rightarrow_{\widehat{G}} u_1 x_1 y_j x_2 u_2.$$

Thus we can reach the same sentential form with G and $\widehat{G}$. If (6.1) is used again later, we can repeat the argument. It follows then, by induction on the number of times the production is applied, that

$$S \xRightarrow{*}_{\widehat{G}} w.$$

Therefore, if $w \in L(G)$, then $w \in L(\widehat{G})$.

By similar reasoning, we can show that if $w \in L(\widehat{G})$, then $w \in L(G)$, completing the proof. ■

Theorem 6.1 is a simple and quite intuitive substitution rule: A production $A \rightarrow x_1 B x_2$ can be eliminated from a grammar if we put in its place the set of productions in which B is replaced by all strings it derives in one step. In this result, it is necessary that A and B be different variables. The case when $A = B$ is partially addressed in Exercises 25 and 26 at the end of this section.

EXAMPLE 6.1

Consider $G = (\{A, B\}, \{a, b, c\}, A, P)$ with productions

$$\begin{aligned} A &\rightarrow a | aaA | abBc, \\ B &\rightarrow abbA | b. \end{aligned}$$

Using the suggested substitution for the variable B , we get the grammar $\widehat{G}$ with productions

$$\begin{aligned} A &\rightarrow a | aaA | ababbAc | abbc, \\ B &\rightarrow abbA | b. \end{aligned}$$

The new grammar $\widehat{G}$ is equivalent to G . The string $aaabbc$ has the derivation

$$A \Rightarrow aaA \Rightarrow aaabBc \Rightarrow aaabbc$$

in G , and the corresponding derivation

$$A \Rightarrow aaA \Rightarrow aaabbc$$

in $\widehat{G}$.

Notice that, in this case, the variable B and its associated productions are still in the grammar even though they can no longer play a part in any derivation. We will next show how such unnecessary productions can be removed from a grammar.

Removing Useless Productions

One invariably wants to remove productions from a grammar that can never take part in any derivation. For example, in the grammar whose entire production set is

$$\begin{aligned} S &\rightarrow aSb \mid \lambda \mid A, \\ A &\rightarrow aA, \end{aligned}$$

the production $S \rightarrow A$ clearly plays no role, as A cannot be transformed into a terminal string. While A can occur in a string derived from S , this can never lead to a sentence. Removing this production leaves the language unaffected and is a simplification by any definition.

DEFINITION 6.1

Let $G = (V, T, S, P)$ be a context-free grammar. A variable $A \in V$ is said to be **useful** if and only if there is at least one $w \in L(G)$ such that

$$S \xRightarrow{*} xAy \xRightarrow{*} w, \tag{6.2}$$

with x, y in $(V \cup T)^*$. In words, a variable is useful if and only if it occurs in at least one derivation. A variable that is not useful is called **useless**. A production is useless if it involves any useless variable.

EXAMPLE 6.2

A variable may be useless because there is no way of getting a terminal string from it. The case just mentioned is of this kind. Another reason

a variable may be useless is shown in the next grammar. In a grammar with start symbol S and productions

$$\begin{aligned} S &\rightarrow A, \\ A &\rightarrow aA|\lambda, \\ B &\rightarrow bA, \end{aligned}$$

the variable B is useless and so is the production $B \rightarrow bA$. Although B can derive a terminal string, there is no way we can achieve $S \xRightarrow{*} xBy$.

This example illustrates the two reasons why a variable is useless: either because it cannot be reached from the start symbol or because it cannot derive a terminal string. A procedure for removing useless variables and productions is based on recognizing these two situations. Before we present the general case and the corresponding theorem, let us look at another example.

EXAMPLE 6.3

Eliminate useless symbols and productions from $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$ and $T = \{a, b\}$, with P consisting of

$$\begin{aligned} S &\rightarrow aS|A|C, \\ A &\rightarrow a, \\ B &\rightarrow aa, \\ C &\rightarrow aCb. \end{aligned}$$

First, we identify the set of variables that can lead to a terminal string. Because $A \rightarrow a$ and $B \rightarrow aa$, the variables A and B belong to this set. So does S , because $S \Rightarrow A \Rightarrow a$. However, this argument cannot be made for C , thus identifying it as useless. Removing C and its corresponding productions, we are led to the grammar G_1 with variables $V_1 = \{S, A, B\}$, terminals $T = \{a\}$, and productions

$$\begin{aligned} S &\rightarrow aS|A, \\ A &\rightarrow a, \\ B &\rightarrow aa. \end{aligned}$$

Next we want to eliminate the variables that cannot be reached from the start variable. For this, we can draw a **dependency graph** for the variables. Dependency graphs are a way of visualizing complex relationships and are found in many applications. For context-free grammars, a dependency graph has its vertices labeled with variables,

with an edge between vertices C and D if and only if there is a production of the form

$$C \rightarrow xDy.$$

A dependency graph for V_1 is shown in Figure 6.1. A variable is useful only if there is a path from the vertex labeled S to the vertex labeled with that variable. In our case, Figure 6.1 shows that B is useless. Removing it and the affected productions and terminals, we are led to the final answer $\hat{G} = (\hat{V}, \hat{T}, S, \hat{P})$ with $\hat{V} = \{S, A\}$, $\hat{T} = \{a\}$, and productions

$$\begin{aligned} S &\rightarrow aS|A, \\ A &\rightarrow a. \end{aligned}$$

The formalization of this process leads to a general construction and the corresponding theorem.



FIGURE 6.1

THEOREM 6.2

Let $G = (V, T, S, P)$ be a context-free grammar. Then there exists an equivalent grammar $\hat{G} = (\hat{V}, \hat{T}, S, \hat{P})$ that does not contain any useless variables or productions.

Proof: The grammar $\hat{G}$ can be generated from G by an algorithm consisting of two parts. In the first part we construct an intermediate grammar $G_1 = (V_1, T_2, S, P_1)$ such that V_1 contains only variables A for which

$$A \xRightarrow{*} w \in T^*$$

is possible. The steps in the algorithm are

1. Set V_1 to $\emptyset$.
2. Repeat the following step until no more variables are added to V_1 . For every $A \in V$ for which P has a production of the form

$$A \rightarrow x_1x_2 \cdots x_n, \text{ with all } x_i \text{ in } V_1 \cup T,$$

add A to V_1 .

3. Take P_1 as all the productions in P whose symbols are all in $(V_1 \cup T)$.

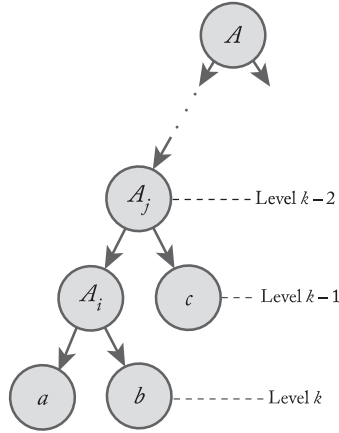


FIGURE 6.2

Clearly this procedure terminates. It is equally clear that if $A \in V_1$, then $A \xRightarrow{*} w \in T^*$ is a possible derivation with G_1 . The remaining issue is whether every A for which $A \xRightarrow{*} w = ab\cdots$ is added to V_1 before the procedure terminates. To see this, consider any such A and look at the partial derivation tree corresponding to that derivation (Figure 6.2). At level k , there are only terminals, so every variable A_i at level $k-1$ will be added to V_1 on the first pass through Step 2 of the algorithm. Any variable at level $k-2$ will then be added to V_1 on the second pass through Step 2. The third time through Step 2, all variables at level $k-3$ will be added, and so on. The algorithm cannot terminate while there are variables in the tree that are not yet in V_1 . Hence A will eventually be added to V_1 .

In the second part of the construction, we get the final answer $\widehat{G}$ from G_1 . We draw the variable dependency graph for G_1 and from it find all variables that cannot be reached from S . These are removed from the variable set, as are the productions involving them. We can also eliminate any terminal that does not occur in some useful production. The result is the grammar $\widehat{G} = (\widehat{V}, \widehat{T}, S, \widehat{P})$.

Because of the construction, $\widehat{G}$ does not contain any useless symbols or productions. Also, for each $w \in L(G)$ we have a derivation

$$S \xRightarrow{*} xAy \xRightarrow{*} w.$$

Since the construction of $\widehat{G}$ retains A and all associated productions, we have everything needed to make the derivation

$$S \xRightarrow{*}_{\widehat{G}} xAy \xRightarrow{*}_{\widehat{G}} w.$$

The grammar $\hat{G}$ is constructed from G by the removal of productions, so that $\hat{P} \subseteq P$. Consequently $L(\hat{G}) \subseteq L(G)$. Putting the two results together, we see that G and $\hat{G}$ are equivalent. ■

Removing λ -Productions

One kind of production that is sometimes undesirable is one in which the right side is the empty string.

DEFINITION 6.2

Any production of a context-free grammar of the form

$$A \rightarrow \lambda$$

is called a **λ -production**. Any variable A for which the derivation

$$A \xRightarrow{*} \lambda \tag{6.3}$$

is possible is called **nullable**.

A grammar may generate a language not containing λ , yet have some λ -productions or nullable variables. In such cases, the λ -productions can be removed.

EXAMPLE 6.4

Consider the grammar

$$\begin{aligned} S &\rightarrow aS_1b, \\ S_1 &\rightarrow aS_1b|\lambda, \end{aligned}$$

with start variable S . This grammar generates the λ -free language $\{a^n b^n : n \geq 1\}$. The λ -production $S_1 \rightarrow \lambda$ can be removed after adding new productions obtained by substituting λ for S_1 where it occurs on the right. Doing this we get the grammar

$$\begin{aligned} S &\rightarrow aS_1b|ab, \\ S_1 &\rightarrow aS_1b|ab. \end{aligned}$$

We can easily show that this new grammar generates the same language as the original one.

In more general situations, substitutions for λ -productions can be made in a similar, although more complicated, manner.

THEOREM 6.3

Let G be any context-free grammar with λ not in $L(G)$. Then there exists an equivalent grammar $\hat{G}$ having no λ -productions.

Proof: We first find the set V_N of all nullable variables of G , using the following steps.

1. For all productions $A \rightarrow \lambda$, put A into V_N .
2. Repeat the following step until no further variables are added to V_N .
For all productions

$$B \rightarrow A_1 A_2 \cdots A_n,$$

where $A_1, A_2, \dots, A_n$ are in V_N , put B into V_N .

Once the set V_N has been found, we are ready to construct $\hat{P}$. To do so, we look at all productions in P of the form

$$A \rightarrow x_1 x_2 \cdots x_m, m \geq 1,$$

where each $x_i \in V \cup T$. For each such production of P , we put into $\hat{P}$ that production as well as all those generated by replacing nullable variables with λ in all possible combinations. For example, if x_i and x_j are both nullable, there will be one production in $\hat{P}$ with x_i replaced with λ , one in which x_j is replaced with λ , and one in which both x_i and x_j are replaced with λ . There is one exception: If all x_i are nullable, the production $A \rightarrow \lambda$ is not put into $\hat{P}$.

The argument that this grammar $\hat{G}$ is equivalent to G is straightforward and will be left to the reader. ■

EXAMPLE 6.5

Find a context-free grammar without λ -productions equivalent to the grammar defined by

$$\begin{aligned} S &\rightarrow ABaC, \\ A &\rightarrow BC, \\ B &\rightarrow b|\lambda, \\ C &\rightarrow D|\lambda, \\ D &\rightarrow d. \end{aligned}$$

From the first step of the construction in Theorem 6.3, we find that the nullable variables are A, B, C . Then, following the second step of the construction, we get

$$\begin{aligned} S &\rightarrow ABaC \mid BaC \mid AaC \mid ABa \mid aC \mid Aa \mid Ba \mid a, \\ A &\rightarrow B \mid C \mid BC, \\ B &\rightarrow b, \\ C &\rightarrow D, \\ D &\rightarrow d. \end{aligned}$$

Removing Unit-Productions

As we have seen in Theorem 5.2, productions in which both sides are a single variable are at times undesirable.

DEFINITION 6.3

Any production of a context-free grammar of the form

$$A \rightarrow B,$$

where $A, B \in V$, is called a **unit-production**.

To remove unit-productions, we use the substitution rule discussed in Theorem 6.1. As the construction in the next theorem shows, this can be done if we proceed with some care.

THEOREM 6.4

Let $G = (V, T, S, P)$ be any context-free grammar without λ -productions. Then there exists a context-free grammar $\widehat{G} = (\widehat{V}, \widehat{T}, S, \widehat{P})$ that does not have any unit-productions and that is equivalent to G .

Proof: Obviously, any unit-production of the form $A \rightarrow A$ can be removed from the grammar without effect, and we need only consider $A \rightarrow B$, where A and B are different variables. At first sight, it may seem that we can use Theorem 6.1 directly with $x_1 = x_2 = \lambda$ to replace

$$A \rightarrow B$$

with

$$A \rightarrow y_1 \mid y_2 \mid \cdots \mid y_n.$$

But this will not always work; in the special case

$$\begin{aligned} A &\rightarrow B, \\ B &\rightarrow A, \end{aligned}$$

the unit-productions are not removed. To get around this, we first find, for each A , all variables B such that

$$A \xRightarrow{*} B. \quad (6.4)$$

We can do this by drawing a dependency graph with an edge (C, D) whenever the grammar has a unit-production $C \rightarrow D$; then (6.4) holds whenever there is a walk between A and B . The new grammar $\hat{G}$ is generated by first putting into $\hat{P}$ all non-unit productions of P . Next, for all A and B satisfying (6.4), we add to $\hat{P}$

$$A \rightarrow y_1 | y_2 | \cdots | y_n,$$

where $B \rightarrow y_1 | y_2 | \cdots | y_n$ is the set of all rules in $\hat{P}$ with B on the left. Note that since $B \rightarrow y_1 | y_2 | \cdots | y_n$ is taken from $\hat{P}$, none of the y_i can be a single variable, so that no unit-productions are created by the last step.

To show that the resulting grammar is equivalent to the original one, we can follow the same line of reasoning as in Theorem 6.1. ■

EXAMPLE 6.6

Remove all unit-productions from

$$\begin{aligned} S &\rightarrow Aa|B, \\ B &\rightarrow A|bb, \\ A &\rightarrow a|bc|B. \end{aligned}$$

The dependency graph for the unit-productions is given in Figure 6.3; we see from it that $S \xRightarrow{*} A$, $S \xRightarrow{*} B$, $B \xRightarrow{*} A$, and $A \xRightarrow{*} B$. Hence, we add to the original non-unit productions

$$\begin{aligned} S &\rightarrow Aa, \\ A &\rightarrow a|bc, \\ B &\rightarrow bb, \end{aligned}$$

the new rules

$$\begin{aligned} S &\rightarrow a|bc|bb, \\ A &\rightarrow bb, \\ B &\rightarrow a|bc, \end{aligned}$$

to obtain the equivalent grammar

$$S \rightarrow a|bc|bb|Aa,$$

$$A \rightarrow a|bb|bc,$$

$$B \rightarrow a|bb|bc.$$

Note that the removal of the unit-productions has made B and the associated productions useless.

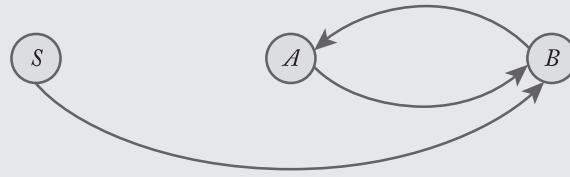


FIGURE 6.3

We can put all these results together to show that grammars for context-free languages can be made free of useless productions, λ -productions, and unit-productions.

THEOREM 6.5

Let L be a context-free language that does not contain λ . Then there exists a context-free grammar that generates L and that does not have any useless productions, λ -productions, or unit-productions.

Proof: The procedures given in Theorems 6.2, 6.3, and 6.4 remove these kinds of productions in turn. The only point that needs consideration is that the removal of one type of production may introduce productions of another type; for example, the procedure for removing λ -productions can create new unit-productions. Also, Theorem 6.4 requires that the grammar have no λ -productions. But note that the removal of unit-productions does not create λ -productions (Exercise 18 at the end of this section), and the removal of useless productions does not create λ -productions or unit-productions (Exercise 19 at the end of this section). Therefore, we can remove all undesirable productions using the following sequence of steps:

1. Remove λ -productions.
2. Remove unit-productions.
3. Remove useless productions.

The result will then have none of these productions, and the theorem is proved. ■

INTRODUCTORY EXERCISES

1. Remove the λ -productions from the grammar

$$\begin{aligned} S &\rightarrow aSSS, \\ S &\rightarrow a|\lambda. \end{aligned}$$

2. Remove all unit-productions from the grammar

$$\begin{aligned} S &\rightarrow aS|A, \\ A &\rightarrow B, \\ B &\rightarrow bb. \end{aligned}$$

3. Show that the grammar

$$\begin{aligned} S &\rightarrow aSABaa, \\ A &\rightarrow aAB, \\ B &\rightarrow bb \end{aligned}$$

and the grammar

$$\begin{aligned} S &\rightarrow aSAbbaa, \\ A &\rightarrow aAbb \end{aligned}$$

are equivalent.

EXERCISES

1. Eliminate the variable B from the grammar

$$\begin{aligned} S &\rightarrow aSB|bB, \\ B &\rightarrow aA|b. \end{aligned}$$

2. Show that the two grammars

$$\begin{aligned} S &\rightarrow abAB|ba, \\ A &\rightarrow aaa, \\ B &\rightarrow aA|bb \end{aligned}$$

and

$$\begin{aligned} S &\rightarrow abAaA|abAbb|ba, \\ A &\rightarrow aaa \end{aligned}$$

are equivalent.

3. Complete the proof of Theorem 6.1 by showing that

$$S \Rightarrow_{\hat{G}}^* w$$

implies

$$S \Rightarrow_G^* w.$$

4. In Example 6.1, show a derivation tree for the string *ababbac*, using both the original and the modified grammar.
5. In Theorem 6.1, why is it necessary to assume that *A* and *B* are different variables?
6. Eliminate all useless productions from the grammar

$$\begin{aligned} S &\rightarrow aS|AB|\lambda, \\ A &\rightarrow bA, \\ B &\rightarrow AA. \end{aligned}$$

What language does this grammar generate?

7. Eliminate useless productions from

$$\begin{aligned} S &\rightarrow a|aA|B|C, \\ A &\rightarrow aB|\lambda, \\ B &\rightarrow Aa, \\ C &\rightarrow cCD, \\ D &\rightarrow ddd|Cd. \end{aligned}$$

8. Eliminate all λ -productions from

$$\begin{aligned} S &\rightarrow aSSS, \\ S &\rightarrow bb|\lambda. \end{aligned}$$

9. Eliminate all λ -productions from

$$\begin{aligned} S &\rightarrow AaB|aaB, \\ A &\rightarrow \lambda, \\ B &\rightarrow bbA|\lambda. \end{aligned}$$

10. Remove all unit-productions, all useless productions, and all λ -productions from the grammar

$$\begin{aligned} S &\rightarrow aA|aBB, \\ A &\rightarrow aaA|\lambda, \\ B &\rightarrow bB|bbC, \\ C &\rightarrow B. \end{aligned}$$

What language does this grammar generate?

11. Eliminate all unit-productions from the grammar in Exercise 7.

12. Complete the proof of Theorem 6.3.
13. Complete the proof of Theorem 6.4.
14. Use the construction in Theorem 6.3 to remove λ -productions from the grammar in Example 5.4. What language does the resulting grammar generate?
15. Consider the grammar G with productions

$$\begin{aligned} S &\rightarrow A|B, \\ A &\rightarrow \lambda, \\ B &\rightarrow aBb, \\ B &\rightarrow b. \end{aligned}$$

Construct a grammar $\hat{G}$ by applying the algorithm in Theorem 6.3 to G . What is the difference between $L(G)$ and $L(\hat{G})$?

16. Suppose that G is a context-free grammar for which $\lambda \in L(G)$. Show that if we apply the construction in Theorem 6.3, we obtain a new grammar $\hat{G}$ such that $L(\hat{G}) = L(G) - \{\lambda\}$.
17. Give an example of a situation in which the removal of λ -productions introduces previously nonexistent unit-productions.
18. Let G be a grammar without λ -productions, but possibly with some unit-productions. Show that the construction of Theorem 6.4 does not then introduce any λ -productions.
19. Show that if a grammar has no λ -productions and no unit-productions, then the removal of useless productions by the construction of Theorem 6.2 does not introduce any such productions.
20. Justify the claim made in the proof of Theorem 6.1 that the variable B can be replaced as soon as it appears.
21. Suppose that a context-free grammar $G = (V, T, S, P)$ has a production of the form

$$A \rightarrow xy,$$

where $x, y \in (V \cup T)^+$. Prove that if this rule is replaced by

$$\begin{aligned} A &\rightarrow By, \\ B &\rightarrow x, \end{aligned}$$

where $B \notin V$, then the resulting grammar is equivalent to the original one.

22. Consider the procedure suggested in Theorem 6.2 for the removal of useless productions. Reverse the order of the two parts, first eliminating variables that cannot be reached from S , then removing those that do not yield a terminal string. Does the new procedure still work correctly? If so, prove it. If not, give a counterexample.
23. It is possible to define the term *simplification* precisely by introducing the concept of **complexity** of a grammar. This can be done in many ways; one

of them is through the length of all the strings giving the production rules. For example, we might use

$$\text{complexity}(G) = \sum_{A \rightarrow v \in P} \{1 + |v|\}.$$

Show that the removal of useless productions always reduces the complexity in this sense. What can you say about the removal of λ -productions and unit-productions?

24. A context-free grammar G is said to be minimal for a given language L if $\text{complexity}(G) \leq \text{complexity}(\hat{G})$ for any $\hat{G}$ generating L . Show by example that the removal of useless productions does not necessarily produce a minimal grammar.
25. Prove the following result. Let $G = (V, T, S, P)$ be a context-free grammar. Divide the set of productions whose left sides are some given variable (say, A), into two disjoint subsets

$$\begin{aligned} A &\rightarrow Ax_1 | Ax_2 | \cdots | Ax_n, \\ A &\rightarrow y_1 | y_2 | \cdots | y_m, \end{aligned}$$

where x_i, y_i are in $(V \cup T)^*$, but A is not a prefix of any y_i . Consider the grammar $\hat{G} = (V \cup \{Z\}, T, S, \hat{P})$, where $Z \notin V$ and $\hat{P}$ is obtained by replacing all productions that have A on the left by

$$\begin{aligned} A &\rightarrow y_i | y_i Z, \quad i = 1, 2, \dots, m, \\ Z &\rightarrow x_i | x_i Z, \quad i = 1, 2, \dots, n. \end{aligned}$$

Then $L(G) = L(\hat{G})$.

26. Use the result of the preceding exercise to rewrite the grammar

$$\begin{aligned} A &\rightarrow Aa | aBc | \lambda, \\ B &\rightarrow Bb | bc \end{aligned}$$

so that it no longer has productions of the form $A \rightarrow Ax$ or $B \rightarrow Bx$.

27. Prove the following counterpart of Exercise 25. Let the set of productions involving the variable A on the left be divided into two disjoint subsets

$$A \rightarrow x_1 A | x_2 A | \cdots | x_n A$$

and

$$A \rightarrow y_1 | y_2 | \cdots | y_m,$$

where A is not a suffix of any y_i . Show that the grammar obtained by replacing these productions with

$$\begin{aligned} A &\rightarrow y_i | Zy_i, \quad i = 1, 2, \dots, m, \\ Z &\rightarrow x_i | Zx_i, \quad i = 1, 2, \dots, n, \end{aligned}$$

is equivalent to the original grammar.

6.2 TWO IMPORTANT NORMAL FORMS

There are many kinds of normal forms that we can establish for context-free grammars. Some of these, because of their wide usefulness, have been studied extensively. We consider two of them briefly.

Chomsky Normal Form

One kind of normal form we can look for is one in which the number of symbols on the right of a production is strictly limited. In particular, we can ask that the string on the right of a production consist of no more than two symbols. One instance of this is the **Chomsky normal form**.

DEFINITION 6.4

A context-free grammar is in Chomsky normal form if all productions are of the form

$$A \rightarrow BC$$

or

$$A \rightarrow a,$$

where A, B, C are in V , and a is in T .

EXAMPLE 6.7

The grammar

$$\begin{aligned} S &\rightarrow AS|a, \\ A &\rightarrow SA|b \end{aligned}$$

is in Chomsky normal form. The grammar

$$\begin{aligned} S &\rightarrow AS|AAS, \\ A &\rightarrow SA|aa \end{aligned}$$

is not; both productions $S \rightarrow AAS$ and $A \rightarrow aa$ violate the conditions of Definition 6.4.

THEOREM 6.6

Any context-free grammar $G = (V, T, S, P)$ with $\lambda \notin L(G)$ has an equivalent grammar $\hat{G} = (\hat{V}, \hat{T}, S, \hat{P})$ in Chomsky normal form.

Proof: Because of Theorem 6.5, we can assume without loss of generality that G has no λ -productions and no unit-productions. The construction of $\hat{G}$ will be done in two steps.

Step 1: Construct a grammar $G_1 = (V_1, T, S, P_1)$ from G by considering all productions in P in the form

$$A \rightarrow x_1 x_2 \cdots x_n, \quad (6.5)$$

where each x_i is a symbol either in V or in T . If $n = 1$, then x_1 must be a terminal since we have no unit-productions. In this case, put the production into P_1 . If $n \geq 2$, introduce new variables B_a for each $a \in T$. For each production of P in the form (6.5) we put into P_1 the production

$$A \rightarrow C_1 C_2 \cdots C_n,$$

where

$$C_i = x_i \text{ if } x_i \text{ is in } V,$$

and

$$C_i = B_a \text{ if } x_i = a.$$

For every B_a we also put into P_1 the production

$$B_a \rightarrow a.$$

This part of the algorithm removes all terminals from productions whose right side has length greater than one, replacing them with newly introduced variables. At the end of this step we have a grammar G_1 , all of whose productions have the form

$$A \rightarrow a, \quad (6.6)$$

or

$$A \rightarrow C_1 C_2 \cdots C_n, \quad (6.7)$$

where $C_i \in V_1$.

It is an easy consequence of Theorem 6.1 that

$$L(G_1) = L(G).$$

Step 2: In the second step, we introduce additional variables to reduce the length of the right sides of the productions where necessary. First we put all productions of the form (6.6) as well as all the productions of the form

(6.7) with $n = 2$ into $\hat{P}$. For $n > 2$, we introduce new variables $D_1, D_2, \dots$ and put into $\hat{P}$ the productions

$$\begin{aligned} A &\rightarrow C_1 D_1, \\ D_1 &\rightarrow C_2 D_2, \\ &\vdots \\ D_{n-2} &\rightarrow C_{n-1} C_n. \end{aligned}$$

Obviously, the resulting grammar $\hat{G}$ is in Chomsky normal form. Repeated applications of Theorem 6.1 will show that $L(G_1) = L(\hat{G})$, so that

$$L(\hat{G}) = L(G).$$

This somewhat informal argument can easily be made more precise. We will leave this to the reader. ■

EXAMPLE 6.8

Convert the grammar with productions

$$\begin{aligned} S &\rightarrow ABa, \\ A &\rightarrow aab, \\ B &\rightarrow Ac \end{aligned}$$

to Chomsky normal form.

As required by the construction of Theorem 6.6, the grammar does not have any λ -productions or any unit-productions.

In Step 1, we introduce new variables B_a, B_b, B_c and use the algorithm to get

$$\begin{aligned} S &\rightarrow ABB_a, \\ A &\rightarrow B_a B_a B_b, \\ B &\rightarrow AB_c, \\ B_a &\rightarrow a, \\ B_b &\rightarrow b, \\ B_c &\rightarrow c. \end{aligned}$$

In the second step, we introduce additional variables to get the first two productions into normal form, and we get the final result

$$\begin{aligned} S &\rightarrow AD_1, \\ D_1 &\rightarrow BB_a, \\ A &\rightarrow B_aD_2, \\ D_2 &\rightarrow B_aB_b, \\ B &\rightarrow AB_c, \\ B_a &\rightarrow a, \\ B_b &\rightarrow b, \\ B_c &\rightarrow c. \end{aligned}$$

Greibach Normal Form

Another useful grammatical form is the **Greibach normal form**. Here we put restrictions not on the length of the right sides of a production, but on the positions in which terminals and variables can appear. Arguments justifying Greibach normal form are a little complicated and not very transparent. Similarly, constructing a grammar in Greibach normal form equivalent to a given context-free grammar is tedious. We therefore deal with this matter very briefly. Nevertheless, Greibach normal form has many theoretical and practical consequences.

DEFINITION 6.5

A context-free grammar is said to be in Greibach normal form if all productions have the form

$$A \rightarrow ax,$$

where $a \in T$ and $x \in V^*$.

If we compare this with Definition 5.4, we see that the form $A \rightarrow ax$ is common to both Greibach normal form and s -grammars, but Greibach normal form does not carry the restriction that the pair (A, a) occur at most once. This additional freedom gives Greibach normal form a generality not possessed by s -grammars.

If a grammar is not in Greibach normal form, we may be able to rewrite it in this form with some of the techniques encountered above. Here are two simple examples.

EXAMPLE 6.9

The grammar

$$\begin{aligned} S &\rightarrow AB, \\ A &\rightarrow aA \mid bB \mid b, \\ B &\rightarrow b \end{aligned}$$

is not in Greibach normal form. However, using the substitution given by Theorem 6.1, we immediately get the equivalent grammar

$$\begin{aligned} S &\rightarrow aAB \mid bBB \mid bB, \\ A &\rightarrow aA \mid bB \mid b, \\ B &\rightarrow b, \end{aligned}$$

which is in Greibach normal form.

EXAMPLE 6.10

Convert the grammar

$$S \rightarrow abSb \mid aa$$

into Greibach normal form.

Here we can use a device similar to the one introduced in the construction of Chomsky normal form. We introduce new variables A and B that are essentially synonyms for a and b , respectively. Substituting for the terminals with their associated variables leads to the equivalent grammar

$$\begin{aligned} S &\rightarrow aBSB \mid aA, \\ A &\rightarrow a, \\ B &\rightarrow b, \end{aligned}$$

which is in Greibach normal form.

In general, though, neither the conversion of a given grammar to Greibach normal form nor the proof that this can always be done is a simple matter. We introduce Greibach normal form here because it will simplify the technical discussion of an important result in the next chapter. However, from a conceptual viewpoint, Greibach normal form plays no further role in our discussion, so we only quote the following general result without proof.

THEOREM 6.7

For every context-free grammar G with $\lambda \notin L(G)$, there exists an equivalent grammar $\hat{G}$ in Greibach normal form.

INTRODUCTORY EXERCISES

1. Convert the grammar

$$S \rightarrow aSb|ab$$

into Chomsky normal form.

2. Convert the grammar

$$S \rightarrow aSb|ab|c$$

into Greibach normal form. Then show the derivation of $w = aaacbbb$ with this grammar.

3. Convert the grammar

$$\begin{aligned} S &\rightarrow AB, \\ A &\rightarrow a|aS, \\ B &\rightarrow b \end{aligned}$$

into Greibach normal form.

EXERCISES

- Convert the grammar $S \rightarrow aSS|a|b$ into Chomsky normal form.
- Convert the grammar $S \rightarrow aSb|Sab|ab$ into Chomsky normal form.
- Transform the grammar

$$S \rightarrow aSaaA|A, A \rightarrow abA|bb$$

into Chomsky normal form.

4. Transform the grammar with productions

$$\begin{aligned} S &\rightarrow baAB, \\ A &\rightarrow bAB|\lambda, \\ B &\rightarrow BAa|A|\lambda \end{aligned}$$

into Chomsky normal form.

5. Convert the grammar

$$\begin{aligned} S &\rightarrow AB|aB, \\ A &\rightarrow abb|\lambda, \\ B &\rightarrow bbA \end{aligned}$$

into Chomsky normal form.

6. Let $G = (V, T, S, P)$ be any context-free grammar without any λ -productions or unit-productions. Let k be the maximum number of symbols on the right of any production in P . Show that there is an equivalent grammar in Chomsky normal form with no more than $(k - 1) |P| + |T|$ production rules.
7. Draw the dependency graph for the grammar in Exercise 5.
8. A linear language is one for which there exists a linear grammar (for a definition, see Example 3.14). Let L be any linear language not containing λ . Show that there exists a grammar $G = (V, T, S, P)$, all of whose productions have one of the forms

$$\begin{aligned} A &\rightarrow aB, \\ A &\rightarrow Ba, \\ A &\rightarrow a, \end{aligned}$$

where $a \in T, A, B \in V$, such that $L = L(G)$.

9. Show that for every context-free grammar $G = (V, T, S, P)$, there is an equivalent one in which all productions have the form

$$A \rightarrow aBC,$$

or

$$A \rightarrow \lambda,$$

where $a \in \Sigma \cup \{\lambda\}, A, B, C \in V$.

10. Convert the grammar

$$S \rightarrow aSb|bSa|a|b|ab$$

into Greibach normal form.

11. Convert the following grammar into Greibach normal form.

$$S \rightarrow aSb|ab|bb.$$

12. Convert the grammar

$$S \rightarrow ab|aS|aaS|aSS$$

into Greibach normal form.

13. Convert the grammar

$$\begin{aligned} S &\rightarrow ABb|a|b, \\ A &\rightarrow aaA|B, \\ B &\rightarrow bAb \end{aligned}$$

into Greibach normal form.

14. Can every linear grammar be converted to a form in which all productions look like $A \rightarrow ax$, where $a \in T$ and $x \in V \cup \{\lambda\}$?
15. A context-free grammar is said to be in two-standard form if all production rules satisfy the following pattern:

$$\begin{aligned} A &\rightarrow aBC, \\ A &\rightarrow aB, \\ A &\rightarrow a, \end{aligned}$$

where $A, B, C \in V$ and $a \in T$.

Convert the grammar $G = (\{S, A, B, C\}, \{a, b\}, S, P)$ with P given as

$$\begin{aligned} S &\rightarrow aSA, \\ A &\rightarrow bABC, \\ B &\rightarrow bb, \\ C &\rightarrow aBC \end{aligned}$$

into two-standard form.

16. Two-standard form is general; for any context-free grammar G with $\lambda \notin L(G)$, there exists an equivalent grammar in two-standard form. Prove this claim.

6.3 A MEMBERSHIP ALGORITHM FOR CONTEXT-FREE GRAMMARS*

In Section 5.2, we claim, without any elaboration, that membership and parsing algorithms for context-free grammars exist that require approximately $|w|^3$ steps to parse a string w . We are now in a position to justify this claim. The algorithm we will describe here is called the CYK algorithm, after its originators J. Cocke, D. H. Younger, and T. Kasami. The algorithm works only if the grammar is in Chomsky normal form and succeeds by breaking one problem into a sequence of smaller ones in the following way. Assume that we have a grammar $G = (V, T, S, P)$ in Chomsky normal form and a string

$$w = a_1 a_2 \cdots a_n.$$

We define substrings

$$w_{ij} = a_i \cdots a_j,$$

and subsets of V

$$V_{ij} = \left\{ A \in V : A \xRightarrow{*} w_{ij} \right\}.$$

Clearly, $w \in L(G)$ if and only if $S \in V_{1n}$.

To compute V_{ij} , observe that $A \in V_{ii}$ if and only if G contains a production $A \rightarrow a_i$. Therefore, V_{ii} can be computed for all $1 \leq i \leq n$ by inspection of w and the productions of the grammar. To continue, notice that for $j > i$, A derives w_{ij} if and only if there is a production $A \rightarrow BC$, with $B \xRightarrow{*} w_{ik}$ and $C \xRightarrow{*} w_{k+1j}$ for some k with $i \leq k, k < j$. In other words,

$$V_{ij} = \bigcup_{k \in \{i, i+1, \dots, j-1\}} \{A : A \rightarrow BC, \text{ with } B \in V_{ik}, C \in V_{k+1, j}\}. \quad (6.8)$$

An inspection of the indices in (6.8) shows that it can be used to compute all the V_{ij} if we proceed in the sequence

1. Compute $V_{11}, V_{22}, \dots, V_{nn}$,
2. Compute $V_{12}, V_{23}, \dots, V_{n-1, n}$,
3. Compute $V_{13}, V_{24}, \dots, V_{n-2, n}$,

and so on.

EXAMPLE 6.11

Determine whether the string $w = aabbb$ is in the language generated by the grammar

$$\begin{aligned} S &\rightarrow AB, \\ A &\rightarrow BB|a, \\ B &\rightarrow AB|b. \end{aligned}$$

First note that $w_{11} = a$, so V_{11} is the set of all variables that immediately derive a , that is, $V_{11} = \{A\}$. Since $w_{22} = a$, we also have $V_{22} = \{A\}$ and, similarly,

$$V_{11} = \{A\}, V_{22} = \{A\}, V_{33} = \{B\}, V_{44} = \{B\}, V_{55} = \{B\}.$$

Now we use (6.8) to get

$$V_{12} = \{A : A \rightarrow BC, B \in V_{11}, C \in V_{22}\}.$$

Since $V_{11} = \{A\}$ and $V_{22} = \{A\}$, the set consists of all variables that occur on the left side of a production whose right side is AA . Since there are none, V_{12} is empty. Next,

$$V_{23} = \{A : A \rightarrow BC, B \in V_{22}, C \in V_{33}\},$$

so the required right side is AB , and we have $V_{23} = \{S, B\}$. A straightforward argument along these lines then gives

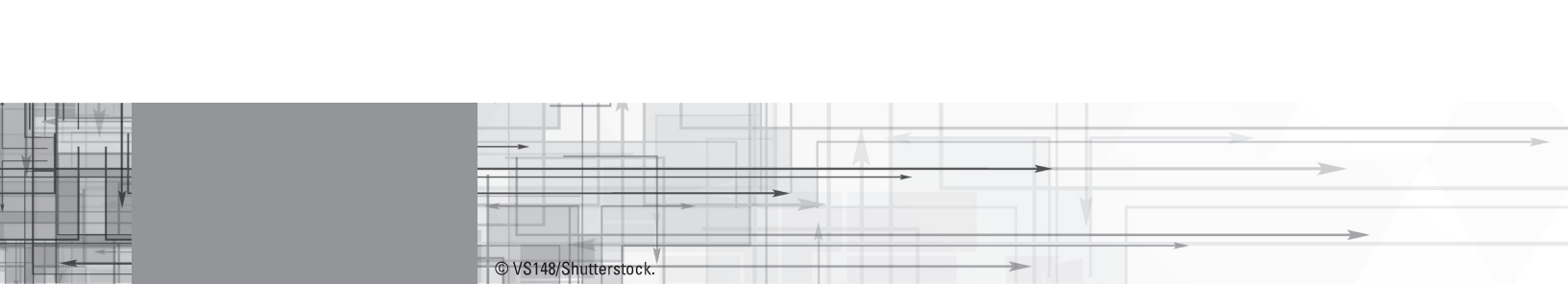
$$\begin{aligned} V_{12} &= \emptyset, V_{23} = \{S, B\}, V_{34} = \{A\}, V_{45} = \{A\}, \\ V_{13} &= \{S, B\}, V_{24} = \{A\}, V_{35} = \{S, B\}, \\ V_{14} &= \{A\}, V_{25} = \{S, B\}, \\ V_{15} &= \{S, B\}, \end{aligned}$$

so that $w \in L(G)$.

The CYK algorithm, as described here, determines membership for any language generated by a grammar in Chomsky normal form. With some additions to keep track of how the elements of V_{ij} are derived, it can be converted into a parsing method. To see that the CYK membership algorithm requires $O(n^3)$ steps, notice that exactly $n(n+1)/2$ sets of V_{ij} have to be computed. Each involves the evaluation of at most n terms in (6.8), so the claimed result follows.

EXERCISES

1. Use the CYK algorithm to determine whether the strings abb , bbb , $aabba$, and $abbbb$ are in the language generated by the grammar in Example 6.11.
2. Use the CYK algorithm to find a parsing of the string aab , using the grammar of Example 6.11.
3. Use the approach employed in Exercise 2 to show how the CYK membership algorithm can be made into a parsing method.
4. Use the CYK method to determine if the string $w = aaabbbb$ is in the language generated by the grammar $S \rightarrow aSb|b$.



CHAPTER

7

PUSHDOWN AUTOMATA

CHAPTER SUMMARY

In the discussion of regular languages, we saw that there were several ways of exploring regular languages: finite automata, regular grammars, and regular expressions. Having defined context-free languages via context-free grammars, we now ask if there are other options for context-free languages. It turns out that there is no analog of regular expressions, but that *pushdown automata* are the automata associated with context-free languages.

Pushdown automata are essentially finite automata with a stack as storage. Since a stack by definition has infinite length, this overcomes the limitation on finite automata arising from a bounded memory.

Pushdown automata are equivalent to context-free grammars, as long as we allow them to be nondeterministic. We can also define deterministic pushdown automata, but the language family associated with them is a proper subset of the context-free languages.

The description of context-free languages by means of context-free grammars is convenient, as illustrated by the use of BNF in programming language definition. The next question is whether there is a class of automata that can be associated with context-free languages. As we have seen, finite automata cannot recognize all context-free languages.

Intuitively, we understand that this is because finite automata have strictly finite memories, whereas the recognition of a context-free language may require storing an unbounded amount of information. For example, when scanning a string from the language $L = \{a^n b^n : n \geq 0\}$, we must not only check that all a 's precede the first b , we must also count the number of a 's. Since n is unbounded, this counting cannot be done with a finite memory. We want a machine that can count without limit. But as we see from other examples, such as $\{ww^R\}$, we need more than unlimited counting ability: We need the ability to store and match a sequence of symbols in reverse order. This suggests that we might try a stack as a storage mechanism, allowing unbounded storage that is restricted to operating like a stack. This gives us a class of machines called **pushdown automata (pda)**.

In this chapter, we explore the connection between pushdown automata and context-free languages. We first show that if we allow pushdown automata to act nondeterministically, we get a class of automata that accepts exactly the family of context-free languages. But we will also see that here there is no longer an equivalence between the deterministic and nondeterministic versions. The class of deterministic pushdown automata defines a new family of languages, the deterministic context-free languages, forming a proper subset of the context-free languages. Since this is an important family for the treatment of programming languages, we conclude the chapter with a brief introduction to the grammars associated with deterministic context-free languages.

7.1 NONDETERMINISTIC PUSHDOWN AUTOMATA

A schematic representation of a pushdown automaton is given in Figure 7.1. Each move of the control unit reads a symbol from the input file, while at the same time changing the contents of the stack through the usual

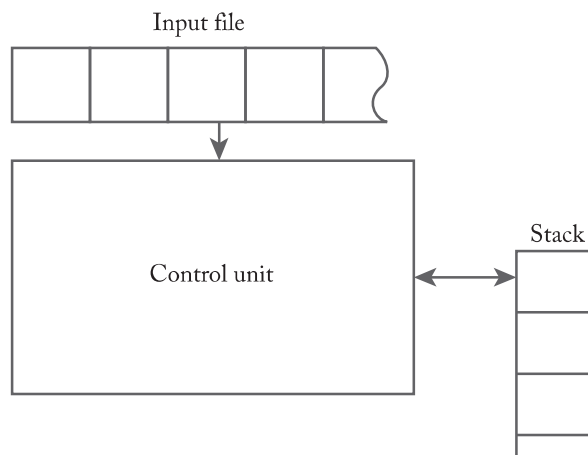


FIGURE 7.1

stack operations. Each move of the control unit is determined by the current input symbol as well as by the symbol currently on top of the stack. The result of the move is a new state of the control unit and a change in the top of the stack.

Definition of a Pushdown Automaton

Formalizing this intuitive notion gives us a precise definition of a pushdown automaton.

DEFINITION 7.1

A **nondeterministic pushdown acceptor (npda)** is defined by the septuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z, F),$$

where

Q is a finite set of internal states of the control unit,

Σ is the input alphabet,

Γ is a finite set of symbols called the **stack alphabet**,

$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow \text{set of finite subsets of } Q \times \Gamma^*$ is the transition function,

$q_0 \in Q$ is the initial state of the control unit,

$z \in \Gamma$ is the **stack start symbol**,

$F \subseteq Q$ is the set of final states.

The complicated formal appearance of the domain and range of δ merits a closer examination. The arguments of δ are the current state of the control unit, the current input symbol, and the current symbol on top of the stack. The result is a set of pairs (q, x) , where q is the next state of the control unit and x is a string that is put on top of the stack in place of the single symbol there before. Note that the second argument of δ may be λ , indicating that a move that does not consume an input symbol is possible. We will call such a move a λ -transition. Note also that δ is defined so that it needs a stack symbol; no move is possible if the stack is empty. Finally, the requirement that the elements of the range of δ be a finite subset is necessary because $Q \times \Gamma^*$ is an infinite set and therefore has infinite subsets. While an npda may have several choices for its moves, this choice must be restricted to a finite set of possibilities.

EXAMPLE 7.1

Suppose the set of transition rules of an npda contains

$$\delta(q_1, a, b) = \{(q_2, cd), (q_3, \lambda)\}.$$

If at any time the control unit is in state q_1 , the input symbol read is a , and the symbol on top of the stack is b , then one of two things can happen: (1) the control unit goes into state q_2 and the string cd replaces b on top of the stack, or (2) the control unit goes into state q_3 with the symbol b removed from the top of the stack. In our notation we assume that the insertion of a string into a stack is done symbol by symbol, starting at the right end of the string.

EXAMPLE 7.2

Consider an npda with

$$Q = \{q_0, q_1, q_2, q_3\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{0, 1\},$$

$$z = 0,$$

$$F = \{q_3\},$$

with initial state q_0 and

$$\delta(q_0, a, 0) = \{(q_1, 10), (q_3, \lambda)\},$$

$$\delta(q_0, \lambda, 0) = \{(q_3, \lambda)\},$$

$$\delta(q_1, a, 1) = \{(q_1, 11)\},$$

$$\delta(q_1, b, 1) = \{(q_2, \lambda)\},$$

$$\delta(q_2, b, 1) = \{(q_2, \lambda)\},$$

$$\delta(q_2, \lambda, 0) = \{(q_3, \lambda)\}.$$

What can we say about the action of this automaton?

First, notice that transitions are not specified for all possible combinations of input and stack symbols. For instance, there is no entry given for $\delta(q_0, b, 0)$. The interpretation of this is the same that we used for nondeterministic finite automata: An unspecified transition is to the null set and represents a dead configuration for the npda.

The crucial transitions are

$$\delta(q_1, a, 1) = \{(q_1, 11)\},$$

which adds a 1 to the stack when an a is read, and

$$\delta(q_2, b, 1) = \{(q_2, \lambda)\},$$

which removes a 1 when a b is encountered. These two steps count the number of a 's and match that count against the number of b 's. The control unit is in state q_1 until the first b is encountered at which time it goes into state q_2 . This ensures that no b precedes the last a . After analyzing the remaining transitions, we see that the npda will end in the final state q_3 if and only if the input string is in the language

$$L = \{a^n b^n : n \geq 0\} \cup \{a\}.$$

As an analogy with finite automata, we might say that the npda accepts the above language. Of course, before making such a claim, we must define what we mean by an npda accepting a language.

We can also use transition graphs to represent npda's. In this representation we label the edges of the graph with three things: the current input symbol, the symbol at the top of the stack, and the string that replaces the top of the stack.

EXAMPLE 7.3

The npda in Example 7.2 is represented by the transition graph in Figure 7.2.

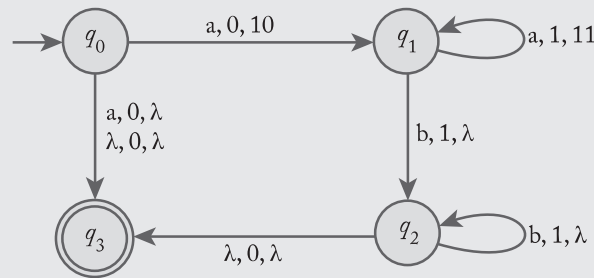


FIGURE 7.2

While transition graphs are convenient for describing npda's, they are less useful for making arguments. The fact that we have to keep track, not only of the internal states, but also of the stack contents, limits the usefulness of transition graphs for formal reasoning. Instead, we introduce a succinct notation for describing the successive configurations of an npda

during the processing of a string. The relevant factors at any time are the current state of the control unit, the unread part of the input string, and the current contents of the stack. Together these completely determine all the possible ways in which the npda can proceed. The triplet

$$(q, w, u),$$

where q is the state of the control unit, w is the unread part of the input string, and u is the stack contents (with the leftmost symbol indicating the top of the stack), is called an **instantaneous description** of a pushdown automaton. A move from one instantaneous description to another will be denoted by the symbol $\vdash$; thus

$$(q_1, aw, bx) \vdash (q_2, w, yx)$$

is possible if and only if

$$(q_2, y) \in \delta(q_1, a, b).$$

Moves involving an arbitrary number of steps will be denoted by $\vdash^*$. The expression

$$(q_1, w_1, x_1) \vdash^* (q_2, w_2, x_2)$$

indicates a possible configuration change over a number of steps.¹

On occasions where several automata are under consideration we will use $\vdash_M$ to emphasize that the move is made by the particular automaton M .

The Language Accepted by a Pushdown Automaton

DEFINITION 7.2

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ be a nondeterministic pushdown automaton. The language accepted by M is the set

$$L(M) = \left\{ w \in \Sigma^* : (q_0, w, z) \vdash_M^* (p, \lambda, u), p \in F, u \in \Gamma^* \right\}.$$

In words, the language accepted by M is the set of all strings that can put M into a final state at the end of the string. The final stack content u is irrelevant to this definition of acceptance.

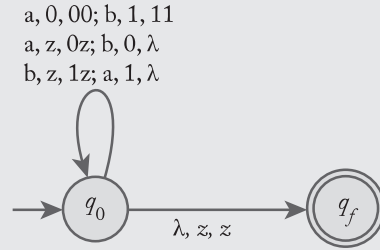
¹Because of the nondeterminism, such a change is of course not necessary.

EXAMPLE 7.4

Construct an npda for the language

$$L = \{w \in \{a, b\}^* : n_a(w) = n_b(w)\}.$$

As in Example 7.2, the solution to this problem involves counting the number of a 's and b 's, which is easily done with a stack. Here we need not even worry about the order of the a 's and b 's. We can insert a counter symbol, say 0, into the stack whenever an a is read, then pop one counter symbol from the stack when a b is found. The only difficulty with this is that if there is a prefix of w with more b 's than a 's, we will not find a 0 to use. But this is easy to fix; we can use a negative counter symbol, say 1, for counting the b 's that are to be matched against a 's later. The complete solution is given in the transition graph in Figure 7.3.

**FIGURE 7.3**

In processing the string $baab$, the npda makes the moves

$$\begin{aligned} (q_0, baab, z) &\vdash (q_0, aab, 1z) \vdash (q_0, ab, z) \\ &\vdash (q_0, b, 0z) \vdash (q_0, \lambda, z) \vdash (q_f, \lambda, z) \end{aligned}$$

and hence the string is accepted.

EXAMPLE 7.5

To construct an npda for accepting the language

$$L = \{ww^R : w \in \{a, b\}^+\},$$

we use the fact that the symbols are retrieved from a stack in the reverse order of their insertion. When reading the first part of the string, we push consecutive symbols on the stack. For the second part, we compare the current input symbol with the top of the stack,

continuing as long as the two match. Since symbols are retrieved from the stack in reverse of the order in which they were inserted, a complete match will be achieved if and only if the input is of the form ww^R .

An apparent difficulty with this suggestion is that we do not know the middle of the string, that is, where w ends and w^R starts. But the nondeterministic nature of the automaton helps us with this; the npda correctly guesses where the middle is and switches states at that point. A solution to the problem is given by $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$, where

$$Q = \{q_0, q_1, q_2\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{a, b, z\},$$

$$F = \{q_2\}.$$

The transition function can be visualized as having several parts: a set to push w on the stack,

$$\delta(q_0, a, a) = \{(q_0, aa)\},$$

$$\delta(q_0, b, a) = \{(q_0, ba)\},$$

$$\delta(q_0, a, b) = \{(q_0, ab)\},$$

$$\delta(q_0, b, b) = \{(q_0, bb)\},$$

$$\delta(q_0, a, z) = \{(q_0, az)\},$$

$$\delta(q_0, b, z) = \{(q_0, bz)\},$$

a set to guess the middle of the string, where the npda switches from state q_0 to q_1 ,

$$\delta(q_0, \lambda, a) = \{(q_1, a)\},$$

$$\delta(q_0, \lambda, b) = \{(q_1, b)\},$$

a set to match w^R against the contents of the stack,

$$\delta(q_1, a, a) = \{(q_1, \lambda)\},$$

$$\delta(q_1, b, b) = \{(q_1, \lambda)\},$$

and finally

$$\delta(q_1, \lambda, z) = \{(q_2, z)\},$$

to recognize a successful match.

The sequence of moves in accepting $abba$ is

$$\begin{aligned} (q_0, abba, z) &\vdash (q_0, bba, az) \vdash (q_0, ba, baz) \\ &\vdash (q_1, ba, baz) \vdash (q_1, a, az) \vdash (q_1, \lambda, z) \vdash (q_2, z). \end{aligned}$$

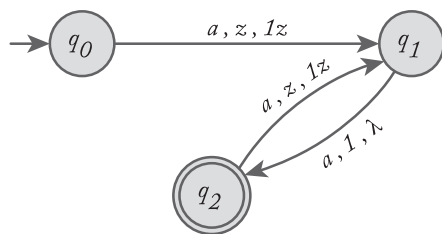
The nondeterministic alternative for locating the middle of the string is taken at the third move. At that stage, the pda has the instantaneous descriptions (q_0, ba, baz) and has two choices for its next move. One is to use $\delta(q_0, b, b) = \{(q_0, bb)\}$ and make the move

$$(q_0, ba, baz) \vdash (q_0, a, bbaz),$$

the second is the one used above, namely $\delta(q_0, \lambda, b) = \{(q_1, b)\}$. Only the latter leads to acceptance of the input.

INTRODUCTORY EXERCISES

1. What transition rule justifies the sequence of instantaneous descriptions $\{q_0, aaabb, 1z\} \vdash \{q_1, aabb, 11z\}$?
2. A pda has a transition rule $\delta(q_0, a, 1) = (q_1, 00)$ and an instantaneous description $(q_0, aw, 1z)$. What is the next instantaneous description?
3. Construct a pda that accepts the language $L = \{a^n b^{n+1}, n \geq 0\}$.
4. Show the sequence of instantaneous descriptions in the processing of $abab$ by the pda in Example 7.4.
5. What is the stack content after the pda in Example 7.4 processes $baaba$?
6. What language is accepted by the pda below?



7. What changes if, in the pda in Exercise 6, you add the rule $\delta(q_2, \lambda, z) = \{(q_0, z)\}$?
8. What changes if, in the pda in Exercise 6, q_0 is also made a final state?

EXERCISES

1. Find a pda that accepts the language $L = \{a^n b^{2n} : n \geq 0\}$.
2. Show the sequence of instantaneous descriptions for the acceptance of $aabbbb$ by the pda in Exercise 1.

3. Construct npda's that accept the following regular languages:

- (a) $L_1 = L(aaa^*bab)$.
- (b) $L_2 = L(aab^*aba^*)$.
- (c) The union of L_1 and L_2 .
- (d) $L_1 - L_2$.
- (e) The intersection of L_1 and L_2 .

4. Find a pda with fewer than four states that accepts the same language as the pda in Example 7.2.

5. Prove that the pda in Example 7.5 does not accept any string not in $\{ww^R\}$.

6. Construct npda's that accept the following languages on $\Sigma = \{a, b, c\}$:

- (a) $L = \{a^n b^{3n} : n \geq 0\}$.
- (b) $L = \{wcw^R : w \in \{a, b\}^*\}$.
- (c) $L = \{a^n b^m c^{n+m} : n \geq 0, m \geq 0\}$.
- (d) $L = \{a^n b^{n+m} c^m : n \geq 0, m \geq 1\}$.
- (e) $L = \{a^3 b^n c^n : n \geq 0\}$.
- (f) $L = \{a^n b^m : n \leq m \leq 3n\}$.
- (g) $L = \{w : n_a(w) = n_b(w) + 1\}$.
- (h) $L = \{w : n_a(w) = 2n_b(w)\}$.
- (i) $L = \{w : n_a(w) + n_b(w) = n_c(w)\}$.
- (j) $L = \{w : 2n_a(w) \leq n_b(w) \leq 3n_a(w)\}$.
- (k) $L = \{w : n_a(w) < n_b(w)\}$.

7. Construct an npda that accepts the language $L = \{a^n b^m : n \geq 0, n \neq m\}$.

8. Find an npda on $\Sigma = \{a, b, c\}$ that accepts the language

$$L = \{w_1 c w_2 : w_1, w_2 \in \{a, b\}^*, w_1 \neq w_2^R\}.$$

9. Find an npda for the concatenation of $L(a^*)$ and the language in Exercise 8.

10. Find an npda for the language $L = \{ab(ab)^n ba(ba)^n : n \geq 0\}$.

11. Is it possible to find a dfa that accepts the same language as the pda

$$M = (\{q_0, q_1\}, \{a, b\}, \{z\}, \delta, q_0, z, \{q_1\}),$$

with

$$\begin{aligned} \delta(q_0, a, z) &= \{(q_1, z)\}, \\ \delta(q_0, b, z) &= \{(q_0, z)\}, \\ \delta(q_1, a, z) &= \{(q_1, z)\}, \\ \delta(q_1, b, z) &= \{(q_0, z)\}? \end{aligned}$$

12. What language is accepted by the pda

$$M = (\{q_0, q_1, q_2, q_3, q_4, q_5\}, \{a, b\}, \{0, 1, z\}, \delta, q_0, z, \{q_5\}),$$

with

$$\begin{aligned}\delta(q_0, b, z) &= \{(q_1, 1z)\}, \\ \delta(q_1, b, 1) &= \{(q_1, 11)\}, \\ \delta(q_2, a, 1) &= \{(q_3, \lambda)\}, \\ \delta(q_3, a, 1) &= \{(q_4, \lambda)\}, \\ \delta(q_4, a, z) &= \{(q_4, z), (q_5, z)\}?\end{aligned}$$

13. What language is accepted by the npda

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, z\}, q_0, z, \{q_2\})$$

with

$$\begin{aligned}\delta(q_0, a, z) &= \{(q_1, a), (q_2, \lambda)\}, \\ \delta(q_1, b, a) &= \{(q_1, b)\}, \\ \delta(q_1, b, b) &= \{(q_1, b)\}, \\ \delta(q_1, a, b) &= \{(q_2, \lambda)\}?\end{aligned}$$

14. What language is accepted by the npda in Example 7.4 if we use $F = \{q_0, q_f\}$?
 15. What language is accepted by the npda in Exercise 13 above if we use $F = \{q_0, q_1, q_2\}$?
 16. Find an npda with no more than two internal states that accepts the language $L(aa^*ba^*)$.
 17. Suppose that in Example 7.2 we replace the given value of $\delta(q_2, \lambda, 0)$ with

$$\delta(q_2, \lambda, 0) = \{(q_0, \lambda)\}.$$

What is the language accepted by this new pda?

18. We can define a restricted npda as one that can increase the length of the stack by at most one symbol in each move, changing Definition 7.1 so that

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow 2^{Q \times (\Gamma \cup \Gamma \cup \{\lambda\})}.$$

The interpretation of this is that the range of δ consists of sets of pairs of the form (q_i, ab) , (q_i, a) , or (q_i, λ) . Show that for every npda M , there exists such a restricted npda $\widehat{M}$ such that $L(M) = L(\widehat{M})$.

19. An alternative to Definition 7.2 for language acceptance is to require the stack to be empty when the end of the input string is reached. Formally, an npda M is said to accept the language $N(M)$ by empty stack if

$$N(M) = \left\{ w \in \Sigma^* : (q_0, w, z) \vdash_M^* (p, \lambda, \lambda) \right\},$$

where p is any element in Q . Show that this notion is effectively equivalent to Definition 7.2, in the sense that for any npda M , there exists an npda $\widehat{M}$ such that $L(M) = N(\widehat{M})$, and vice versa.

7.2 PUSHDOWN AUTOMATA AND CONTEXT-FREE LANGUAGES

In the examples of the previous section, we saw that pushdown automata exist for some of the familiar context-free languages. This is no accident. There is a general relation between context-free languages and nondeterministic pushdown accepters that is established in the next two major results. We will show that for every context-free language there is an npda that accepts it, and conversely, that the language accepted by any npda is context free.

Pushdown Automata for Context-Free Languages

We first show that for every context-free language there is an npda that accepts it. The underlying idea is to construct an npda that can, in some way, carry out a leftmost derivation of any string in the language. To simplify the argument a little, we assume that the language is generated by a grammar in Greibach normal form.

The pda we are about to construct will represent the derivation by keeping the variables in the right part of the sentential form on its stack, while the left part, consisting entirely of terminals, is identical with the input read. We begin by putting the start symbol on the stack. After that, to simulate the application of a production $A \rightarrow ax$, we must have the variable A on top of the stack and the terminal a as the input symbol. The variable on the stack is removed and replaced by the variable string x . What δ should be to achieve this is easy to see. Before we present the general argument, let us look at a simple example.

EXAMPLE 7.6

Construct a pda that accepts the language generated by a grammar with productions

$$S \rightarrow aSbb|a.$$

We first transform the grammar into Greibach normal form, changing the productions to

$$S \rightarrow aSA|a,$$

$$A \rightarrow bB,$$

$$B \rightarrow b.$$

The corresponding automaton will have three states $\{q_0, q_1, q_2\}$, with initial state q_0 and final state q_2 . First, the start symbol S is put on the stack by

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}.$$

The production $S \rightarrow aSA$ will be simulated in the pda by removing S from the stack and replacing it with SA , while reading a from the input. Similarly, the rule $S \rightarrow a$ should cause the pda to read an a while simply removing S . Thus, the two productions are represented in the pda by

$$\delta(q_1, a, S) = \{(q_1, SA), (q_1, \lambda)\}.$$

In an analogous manner, the other productions give

$$\delta(q_1, b, A) = \{(q_1, B)\},$$

$$\delta(q_1, b, B) = \{(q_1, \lambda)\}.$$

The appearance of the stack start symbol on top of the stack signals the completion of the derivation and the pda is put into its final state by

$$\delta(q_1, \lambda, z) = \{(q_2, \lambda)\}.$$

The construction of this example can be adapted to other cases, leading to a general result.

THEOREM 7.1

For any context-free language L , there exists an npda M such that

$$L = L(M).$$

Proof: If L is a λ -free context-free language, there exists a context-free grammar in Greibach normal form for it. Let $G = (V, T, S, P)$ be such a grammar. We then construct an npda that simulates leftmost derivations in this grammar. As suggested, the simulation will be done so that the unprocessed part of the sentential form is in the stack, while the terminal prefix of any sentential form matches the corresponding prefix of the input string.

Specifically, the npda will be

$$M = (\{q_0, q_1, q_f\}, T, V \cup \{z\}, \delta, q_0, z, \{q_f\}),$$

where $z \notin V$. Note that the input alphabet of M is identical with the set of terminals of G and that the stack alphabet contains the set of variables of the grammar.

The transition function will include

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}, \tag{7.1}$$

so that after the first move of M , the stack contains the start symbol S of the derivation. (The stack start symbol z is a marker to allow us to detect the end of the derivation.) In addition, the set of transition rules is such that

$$(q_1, u) \in \delta(q_1, a, A), \quad (7.2)$$

whenever

$$A \rightarrow au$$

is in P . This reads input a and removes the variable A from the stack, replacing it with u . In this way it generates the transitions that allow the pda to simulate all derivations. Finally, we have

$$\delta(q_1, \lambda, z) = \{(q_f, z)\}, \quad (7.3)$$

to get M into a final state.

To show that M accepts any $w \in L(G)$, consider the partial leftmost derivation

$$\begin{aligned} S &\xRightarrow{*} a_1 a_2 \cdots a_n A_1 A_2 \cdots A_m \\ &\Rightarrow a_1 a_2 \cdots a_n b B_1 \cdots B_k A_2 \cdots A_m. \end{aligned}$$

If M is to simulate this derivation, then after reading $a_1 a_2 \cdots a_n$, the stack must contain $A_1 A_2 \cdots A_m$. To take the next step in the derivation, G must have a production

$$A_1 \rightarrow b B_1 \cdots B_k.$$

But the construction is such that then M has a transition rule in which

$$(q_1, B_1 \cdots B_k) \in \delta(q_1, b, A_1),$$

so that the stack now contains $B_1 \cdots B_k A_2 \cdots A_m$ after having read $a_1 a_2 \cdots a_n b$.

A simple induction argument on the number of steps in the derivation then shows that if

$$S \xRightarrow{*} w,$$

then

$$(q_1, w, Sz) \vdash^* (q_1, \lambda, z).$$

Using (7.1) and (7.3) we have

$$(q_0, w, z) \vdash (q_1, w, Sz) \vdash^* (q_1, \lambda, z) \vdash (q_f, \lambda, z),$$

so that $L(G) \subseteq L(M)$.

To prove that $L(M) \subseteq L(G)$, let $w \in L(M)$. Then by definition

$$(q_0, w, z) \vdash^* (q_f, \lambda, u).$$

But there is only one way to get from q_0 to q_1 and only one way from q_1 to q_f . Therefore, we must have

$$(q_1, w, Sz) \vdash^* (q_1, \lambda, z).$$

Now let us write $w = a_1 a_2 a_3 \cdots a_n$. Then the first step in

$$(q_1, a_1 a_2 a_3 \cdots a_n, Sz) \vdash^* (q_1, \lambda, z) \quad (7.4)$$

must be a rule of the form (7.2) to get

$$(q_1, a_1 a_2 a_3 \cdots a_n, Sz) \vdash (q_1, a_2 a_3 \cdots a_n, u_1 z).$$

But then the grammar has a rule of the form $S \rightarrow a_1 u_1$, so that

$$S \Rightarrow a_1 u_1.$$

Repeating this, writing $u_1 = A u_2$, we have

$$(q_1, a_2 a_3 \cdots a_n, A u_2 z) \vdash (q_1, a_3 \cdots a_n, u_3 u_2 z),$$

implying that $A \rightarrow a_2 u_3$ is in the grammar and that

$$S \Rightarrow^* a_1 a_2 u_3 u_2.$$

This makes it quite clear at any point the stack contents (excluding z) are identical with the unmatched part of the sentential form, so that (7.4) implies

$$S \Rightarrow^* a_1 a_2 \cdots a_n.$$

In consequence, $L(M) \subseteq L(G)$, completing the proof if the language does not contain λ .

If $\lambda \in L$, we add to the constructed npda the transition

$$\delta(q_0, \lambda, z) = \{(q_f, z)\}$$

so that the empty string is also accepted. ■

EXAMPLE 7.7

Consider the grammar

$$\begin{aligned} S &\rightarrow aA, \\ A &\rightarrow aABC|bB|a, \\ B &\rightarrow b, \\ C &\rightarrow c. \end{aligned}$$

Since the grammar is already in Greibach normal form, we can use the construction in the previous theorem immediately. In addition to rules

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}$$

and

$$\delta(q_1, \lambda, z) = \{(q_f, z)\},$$

the pda will also have transition rules

$$\begin{aligned}\delta(q_1, a, S) &= \{(q_1, A)\}, \\ \delta(q_1, a, A) &= \{(q_1, ABC), (q_1, \lambda)\}, \\ \delta(q_1, b, A) &= \{(q_1, B)\}, \\ \delta(q_1, b, B) &= \{(q_1, \lambda)\}, \\ \delta(q_1, c, C) &= \{(q_1, \lambda)\}.\end{aligned}$$

The sequence of moves made by M in processing $aaabc$ is

$$\begin{aligned}(q_0, aaabc, z) &\vdash (q_1, aaabc, Sz) \\ &\vdash (q_1, aabc, Az) \\ &\vdash (q_1, abc, ABCz) \\ &\vdash (q_1, bc, BCz) \\ &\vdash (q_1, c, Cz) \\ &\vdash (q_1, \lambda, z) \\ &\vdash (q_f, \lambda, z).\end{aligned}$$

This corresponds to the derivation

$$S \Rightarrow aA \Rightarrow aaABC \Rightarrow aaaBC \Rightarrow aaabC \Rightarrow aaabc.$$

In order to simplify the arguments, the proof in Theorem 7.1 assumed that the grammar was in Greibach normal form. It is not necessary to do this; we can make a similar and only slightly more complicated construction from a general context-free grammar. For example, for productions of the form

$$A \rightarrow Bx,$$

we remove A from the stack and replace it with Bx , but consume no input symbol. For productions of the form

$$A \rightarrow abCx,$$

we must first match the ab in the input against a similar string in the stack and then replace A with Cx . We leave the details of the construction and the associated proof as an exercise.

Context-Free Grammars for Pushdown Automata

The converse of Theorem 7.1 is also true. The construction involved readily suggests itself: Reverse the process in Theorem 7.1 so that the grammar simulates the moves of the pda. This means that the content of the stack should be reflected in the variable part of the sentential form, while the processed input is the terminal prefix of the sentential form. Quite a few details are needed to make this work.

To keep the discussion as simple as possible, we will assume that the npda in question meets the following requirements:

1. It has a single final state q_f that is entered if and only if the stack is empty;
2. With $a \in \Sigma \cup \{\lambda\}$, all transitions must have the form $\delta(q_i, a, A) = \{c_1, c_2, \dots, c_n\}$, where

$$c_i = (q_j, \lambda), \quad (7.5)$$

or

$$c_i = (q_j, BC). \quad (7.6)$$

That is, each move either increases or decreases the stack content by a single symbol.

These restrictions may appear to be very severe, but they are not. It can be shown that for any npda there exists an equivalent one having properties 1 and 2. This equivalence is explored partially in Exercises 18 and 19 in Section 7.1. Here we need to explore it further, but again we will leave the arguments as an exercise (see Exercise 17 at the end of this section). Taking this as given, we now construct a context-free grammar for the language accepted by the npda.

As stated, we want the sentential form to represent the content of the stack. But the configuration of the npda also involves an internal state, and this has to be remembered in the sentential form as well. It is hard to see how this can be done, and the construction we give here is a little tricky.

Suppose for the moment that we can find a grammar whose variables are of the form $(q_i A q_j)$ and whose productions are such that

$$(q_i A q_j) \xRightarrow{*} v,$$

if and only if the npda erases A from the stack while reading v and going from state q_i to state q_j . “Erasing” here means that A and its effects (i.e.,

all the successive strings by which it is replaced) are removed from the stack, bringing the symbol originally below A to the top. If we can find such a grammar, and if we choose $(q_0 z q_f)$ as its start symbol, then

$$(q_0 z q_f) \xRightarrow{*} w$$

if and only if the npda removes z (creating an empty stack) while reading w and going from q_0 to q_f . But this is exactly how the npda accepts w . Therefore, the language generated by the grammar will be identical to the language accepted by the npda.

To construct a grammar that satisfies these conditions, we examine the different types of transitions that can be made by the npda. Since (7.5) involves an immediate erasure of A , the grammar will have a corresponding production

$$(q_i A q_j) \rightarrow a.$$

Productions of type (7.6) generate the set of rules

$$(q_i A q_k) \rightarrow a (q_j B q_l) (q_l C q_k),$$

where q_k and q_l take on all possible values in Q . This is due to the fact that to erase A we first replace it with BC , while reading an a and going from state q_i to q_j . Subsequently, we go from q_j to q_l , erasing B , then from q_l to q_k , erasing C .

In the last step, it may seem that we have added too much, as there may be some states q_l that cannot be reached from q_j while erasing B . This is true, but this does not affect the grammar. The resulting variables $(q_j B q_l)$ are useless variables and do not affect the language accepted by the grammar.

Finally, as a start variable we take $(q_0 z q_f)$, where q_f is the single final state of the npda.

EXAMPLE 7.8

Consider the npda with transitions

$$\begin{aligned}\delta(q_0, a, z) &= \{(q_0, Az)\}, \\ \delta(q_0, a, A) &= \{(q_0, A)\}, \\ \delta(q_0, b, A) &= \{(q_1, \lambda)\}, \\ \delta(q_1, \lambda, z) &= \{(q_2, \lambda)\}.\end{aligned}$$

Using q_0 as the initial state and q_2 as the final state, the npda satisfies condition 1 above, but not 2. To satisfy the latter, we introduce a new state q_3 and an intermediate step in which we first remove the A from

the stack, then replace it in the next move. The new set of transition rules is

$$\begin{aligned}\delta(q_0, a, z) &= \{(q_0, Az)\}, \\ \delta(q_3, \lambda, z) &= \{(q_0, Az)\}, \\ \delta(q_0, a, A) &= \{(q_3, \lambda)\}, \\ \delta(q_0, b, A) &= \{(q_1, \lambda)\}, \\ \delta(q_1, \lambda, z) &= \{(q_2, \lambda)\}.\end{aligned}$$

The last three transitions are of the form (7.5) so that they yield the corresponding productions

$$(q_0 A q_3) \rightarrow a, \quad (q_0 A q_1) \rightarrow b, \quad (q_1 z q_2) \rightarrow \lambda.$$

From the first two transitions we get the set of productions

$$\begin{aligned}(q_0 z q_0) &\rightarrow a(q_0 A q_0)(q_0 z q_0) | a(q_0 A q_1)(q_1 z q_0) | \\ &\quad a(q_0 A q_2)(q_2 z q_0) | a(q_0 A q_3)(q_3 z q_0), \\ (q_0 z q_1) &\rightarrow a(q_0 A q_0)(q_0 z q_1) | a(q_0 A q_1)(q_1 z q_1) | \\ &\quad a(q_0 A q_2)(q_2 z q_1) | a(q_0 A q_3)(q_3 z q_1), \\ (q_0 z q_2) &\rightarrow a(q_0 A q_0)(q_0 z q_2) | a(q_0 A q_1)(q_1 z q_2) | \\ &\quad a(q_0 A q_2)(q_2 z q_2) | a(q_0 A q_3)(q_3 z q_2), \\ (q_0 z q_3) &\rightarrow a(q_0 A q_0)(q_0 z q_3) | a(q_0 A q_1)(q_1 z q_3) | \\ &\quad a(q_0 A q_2)(q_2 z q_3) | a(q_0 A q_3)(q_3 z q_3),\end{aligned}$$

$$\begin{aligned}(q_3 z q_0) &\rightarrow (q_0 A q_0)(q_0 z q_0) | (q_0 A q_1)(q_1 z q_0) | (q_0 A q_2)(q_2 z q_0) | (q_0 A q_3)(q_3 z q_0), \\ (q_3 z q_1) &\rightarrow (q_0 A q_0)(q_0 z q_1) | (q_0 A q_1)(q_1 z q_1) | (q_0 A q_2)(q_2 z q_1) | (q_0 A q_3)(q_3 z q_1), \\ (q_3 z q_2) &\rightarrow (q_0 A q_0)(q_0 z q_2) | (q_0 A q_1)(q_1 z q_2) | (q_0 A q_2)(q_2 z q_2) | (q_0 A q_3)(q_3 z q_2), \\ (q_3 z q_3) &\rightarrow (q_0 A q_0)(q_0 z q_3) | (q_0 A q_1)(q_1 z q_3) | (q_0 A q_2)(q_2 z q_3) | (q_0 A q_3)(q_3 z q_3).\end{aligned}$$

This looks quite complicated, but it can be simplified. A variable that does not occur on the left side of any production must be useless, so we can immediately eliminate $(q_0 A q_0)$ and $(q_0 A q_2)$. Also, by looking at the transition graph of the modified npda, we see that there is no path from q_1 to q_0 , from q_1 to q_1 , from q_1 to q_3 , and from q_2 to q_2 , which makes the associated variables also useless. When we eliminate all these

useless productions, we are left with the much shorter grammar

$$\begin{aligned}
(q_0 A q_3) &\rightarrow a, \\
(q_0 A q_1) &\rightarrow b, \\
(q_1 z q_2) &\rightarrow \lambda, \\
(q_0 z q_0) &\rightarrow a(q_0 A q_3)(q_3 z q_0), \\
(q_0 z q_1) &\rightarrow a(q_0 A q_3)(q_3 z q_1), \\
(q_0 z q_2) &\rightarrow a(q_0 A q_1)(q_1 z q_2) | a(q_0 A q_3)(q_3 z q_2), \\
(q_0 z q_3) &\rightarrow a(q_0 A q_3)(q_3 z q_3), \\
(q_3 z q_0) &\rightarrow (q_0 A q_3)(q_3 z q_0), \\
(q_3 z q_1) &\rightarrow (q_0 A q_3)(q_3 z q_1), \\
(q_3 z q_2) &\rightarrow (q_0 A q_1)(q_1 z q_2) | (q_0 A q_3)(q_3 z q_2), \\
(q_3 z q_3) &\rightarrow (q_0 A q_3)(q_3 z q_3),
\end{aligned}$$

with start variable $(q_0 z q_2)$.

EXAMPLE 7.9

Consider the string $w = aab$. This is accepted by the pda in Example 7.8, with successive configurations

$$\begin{aligned}
(q_0, aab, z) &\vdash (q_0, ab, Az) \\
&\vdash (q_3, b, z) \\
&\vdash (q_0, b, Az) \\
&\vdash (q_1, \lambda, z) \\
&\vdash (q_2, \lambda, \lambda).
\end{aligned}$$

The corresponding derivation with G is

$$\begin{aligned}
(q_0 z q_2) &\Rightarrow a(q_0 A q_3)(q_3 z q_2) \\
&\Rightarrow aa(q_3 z q_2) \\
&\Rightarrow aa(q_0 A q_1)(q_1 z q_2) \\
&\Rightarrow aab(q_1 z q_2) \\
&\Rightarrow aab.
\end{aligned}$$

The steps in the proof of the following theorem will be easier to understand if you notice the correspondence between the successive instantaneous descriptions of the pda and the sentential forms in the

derivation. The first q_i in the leftmost variable of every sentential form is the current state of the pda, while the sequence of middle symbols is the same as the stack content.

Although the construction yields a rather complicated grammar, it can be applied to any pda whose transition rules satisfy the given conditions. This forms the basis for the proof of the general result.

THEOREM 7.2

If $L = L(M)$ for some npda M , then L is a context-free language.

Proof: Assume that $M = (Q, \Sigma, \Gamma, \delta, q_0, z, \{q_f\})$ satisfies conditions 1 and 2 above. We use the suggested construction to get the grammar $G = (V, T, S, P)$, with $T = \Sigma$ and V consisting of elements of the form $(q_i c q_j)$. We will show that the grammar so obtained is such that for all $q_i, q_j, \in Q, A \in \Gamma, X \in \Gamma^*, u, v \in \Sigma^*$,

$$(q_i, uv, AX) \vdash^* (q_j, v, X) \quad (7.7)$$

implies that

$$(q_i A q_j) \Rightarrow^* u,$$

and vice versa.

The first part is to show that, whenever the npda is such that the symbol A and its effects can be removed from the stack while reading u and going from state q_i to q_j , then the variable $(q_i A q_j)$ can derive u . This is not hard to see since the grammar was explicitly constructed to do this. We only need an induction on the number of moves to make this precise.

For the converse, consider a single step in the derivation such as

$$(q_i A q_k) \Rightarrow a (q_j B q_l) (q_l C q_k).$$

Using the corresponding transition for the npda

$$\delta(q_i, a, A) = \{(q_j, BC), \dots\}, \quad (7.8)$$

we see that the A can be removed from the stack, BC put on, reading a , with the control unit going from state q_i to q_j . Similarly, if

$$(q_i A q_j) \Rightarrow a, \quad (7.9)$$

then there must be a corresponding transition

$$\delta(q_i, a, A) = \{(q_j, \lambda)\} \quad (7.10)$$

whereby the A can be popped off the stack. We see from this that the sentential forms derived from $(q_i A q_j)$ define a sequence of possible configurations of the npda by which (7.7) can be achieved.

Note that $(q_i A q_j) \Rightarrow a(q_j B q_l)(q_l C q_k)$ might be possible for some $(q_j B q_l)(q_l C q_k)$ for which there is no corresponding transition of the form (7.8) or (7.10). But, in that case, at least one of the variables on the right will be useless. For all sentential forms leading to a terminal string, the argument given holds.

If we now apply the conclusion to

$$(q_0, w, z) \vdash^* (q_f, \lambda, \lambda),$$

we see that this can be so if and only if

$$(q_0 z q_f) \Rightarrow^* w.$$

Consequently $L(M) = L(G)$. ■

INTRODUCTORY EXERCISES

1. Find an npda for the grammar

$$\begin{aligned} S &\rightarrow aB, \\ B &\rightarrow b, \\ B &\rightarrow bBC, \\ C &\rightarrow c. \end{aligned}$$

2. Convert the grammar

$$\begin{aligned} S &\rightarrow aAB, \\ A &\rightarrow a|aS, \\ B &\rightarrow b \end{aligned}$$

into an equivalent pda.

3. Convert the grammar

$$\begin{aligned} S &\rightarrow aABa, \\ A &\rightarrow a|aS, \\ B &\rightarrow b \end{aligned}$$

into an equivalent pda.

EXERCISES

1. Construct a pda that accepts the language defined by the grammar $S \rightarrow abSb|\lambda$.
2. Construct a pda that accepts the language defined by the grammar $S \rightarrow aSSSab|\lambda$.
3. Construct an npda that accepts the language generated by the grammar

$$S \rightarrow aSbb|abb.$$

4. Show that the pda constructed in Example 7.6 accepts the strings $aabb$ and $aaabbbb$ and that both strings are in the language generated by the given grammar.
5. Prove that the pda in Example 7.6 accepts the language $L = \{a^{n+1}b^{2n} : n \geq 0\}$.
6. Construct an npda that accepts the language generated by the grammar $S \rightarrow aSSS|ba$.
7. Construct an npda corresponding to the grammar

$$S \rightarrow aABB|aAA,$$

$$A \rightarrow aBB|b,$$

$$B \rightarrow bBB|A.$$

8. Construct an npda that will accept the language generated by the grammar $G = (\{S, A\}, \{a, b\}, S, P)$, with productions $S \rightarrow AA|a, A \rightarrow SA|ab$.
9. Show that Theorems 7.1 and 7.2 imply the following. For every npda M , there exists an npda $\widehat{M}$ with at most three states, such that $L(M) = L(\widehat{M})$.
10. Show how the number of states of $\widehat{M}$ in the above exercise can be reduced to two.
11. Find an npda with two states for the language $L = \{a^n b^{n+1} : n \geq 0\}$.
12. Find an npda with two states that accepts $L = \{a^n b^{2n} : n \geq 2\}$.
13. Show that the npda in Example 7.8 accepts $L(aa^*b)$.
14. Show that the grammar in Example 7.8 generates the language $L(aa^*b)$.
15. In Example 7.8, show that the variable $(q_0 z q_1)$ is useless.
16. Find a context-free grammar that generates the language accepted by the npda $M = (\{q_0, q_1\}, \{a, b\}, \{A, z\}, \delta, q_0, z, \{q_1\})$, with transitions

$$\delta(q_0, a, z) = \{(q_0, Az)\},$$

$$\delta(q_0, b, A) = \{(q_0, AA)\},$$

$$\delta(q_0, a, A) = \{(q_1, \lambda)\}.$$

17. Show that for every npda there exists an equivalent one, satisfying conditions 1 and 2 in the preamble to Theorem 7.2.
 18. Give full details of the proof of Theorem 7.2.
 19. Give a construction by which an arbitrary context-free grammar can be used in the proof of Theorem 7.1.
 20. Does the grammar in Example 7.8 still have any useless variables?
-

7.3 DETERMINISTIC PUSHDOWN AUTOMATA AND DETERMINISTIC CONTEXT-FREE LANGUAGES

A **deterministic pushdown acceptor (dpda)** is a pushdown automaton that never has a choice in its move. This can be achieved by a modification of Definition 7.1.

DEFINITION 7.3

A pushdown automaton $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ is said to be deterministic if it is an automaton as defined in Definition 7.1, subject to the restrictions that, for every $q \in Q, a \in \Sigma \cup \{\lambda\}$ and $b \in \Gamma$,

1. $\delta(q, a, b)$ contains at most one element,
2. if $\delta(q, \lambda, b)$ is not empty, then $\delta(q, c, b)$ must be empty for every $c \in \Sigma$.

The first of these conditions simply requires that for any given input symbol and any stack top, at most one move can be made. The second condition is that when a λ -move is possible for some configuration, no input-consuming alternative is available.

It is interesting to note the difference between this definition and the corresponding definition of a deterministic finite automaton. The domain of the transition function is still as in Definition 7.1 rather than $Q \times \Sigma \times \Gamma$ because we want to retain λ -transitions. Since the top of the stack plays a role in determining the next move, the presence of λ -transitions does not automatically imply nondeterminism. Also, some transitions of a dpda may be to the empty set, that is, undefined, so there may be dead configurations. This does not affect the definition; the only criterion for determinism is that at all times at most one possible move exists.

DEFINITION 7.4

A language L is said to be a **deterministic context-free language** if and only if there exists a dpda M such that $L = L(M)$.

EXAMPLE 7.10

The language

$$L = \{a^n b^n : n \geq 0\}$$

is a deterministic context-free language. The pda $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{0, 1\}, \delta, q_0, 0, \{q_0\})$ with

$$\begin{aligned}\delta(q_0, a, 0) &= \{(q_1, 10)\}, \\ \delta(q_1, a, 1) &= \{(q_1, 11)\}, \\ \delta(q_1, b, 1) &= \{(q_2, \lambda)\}, \\ \delta(q_2, b, 1) &= \{(q_2, \lambda)\}, \\ \delta(q_2, \lambda, 0) &= \{(q_0, \lambda)\}\end{aligned}$$

accepts the given language. It satisfies the conditions of Definition 7.3 and is therefore deterministic.

Look now at Example 7.5. The npda there is not deterministic because

$$\delta(q_0, a, a) = \{(q_0, aa)\}$$

and

$$\delta(q_0, \lambda, a) = \{(q_1, a)\}$$

violate condition 2 of Definition 7.3. This, of course, does not imply that the language $\{ww^R\}$ itself is nondeterministic, since there is the possibility of an equivalent dpda. But it is known that the language is indeed not deterministic. From this and the next example we see that, in contrast to finite automata, deterministic and nondeterministic pushdown automata are not equivalent. There are context-free languages that are not deterministic.

EXAMPLE 7.11

Let

$$L_1 = \{a^n b^n : n \geq 0\}$$

and

$$L_2 = \{a^n b^{2n} : n \geq 0\}.$$

An obvious modification of the argument that L_1 is a context-free language shows that L_2 is also context free. The language

$$L = L_1 \cup L_2$$

is context-free as well. This will follow from a general theorem to be presented in the next chapter, but it can easily be made plausible at this point. Let $G_1 = (V_1, T, S_1, P_1)$ and $G_2 = (V_2, T, S_2, P_2)$ be context-free grammars such that $L_1 = L(G_1)$ and $L_2 = L(G_2)$. If we assume that V_1 and V_2 are disjoint and that $S \notin V_1 \cup V_2$, then, combining the two, grammar $G = (V_1 \cup V_2 \cup \{S\}, T, S, P)$, where

$$P = P_1 \cup P_2 \cup \{S \rightarrow S_1 | S_2\},$$

generates $L_1 \cup L_2$. This should be fairly clear at this point, but the details of the argument will be deferred until Chapter 8. Accepting this, we see that L is context free. But L is not a deterministic context-free language. This seems reasonable, since the pda has either to match one b or two against each a , and so has to make an initial choice whether the input is in L_1 or in L_2 . There is no information available at the beginning of the string by which the choice can be made deterministically. Of course, this sort of argument is based on a particular algorithm we have in mind; it may lead us to the correct conjecture but does not prove anything. There is always the possibility of a completely different approach that avoids an initial choice. But it turns out that there is not, and L is indeed nondeterministic. To see this we first establish the following claim. If L were a deterministic context-free language, then

$$\widehat{L} = L \cup \{a^n b^n c^n : n \geq 0\}$$

would be a context-free language. We show the latter by constructing an npda $\widehat{M}$ for $\widehat{L}$, given a dpda M for L .

The idea behind the construction is to add to the control unit of M a similar part in which transitions caused by the input symbol b are replaced with similar ones for input c . This new part of the control unit may be entered after M has read $a^n b^n$. Since the second part responds to c^n in the same way as the first part does to b^n , the process that recognizes $a^n b^{2n}$ now also accepts $a^n b^n c^n$. Figure 7.4 describes the construction graphically; a formal argument follows.

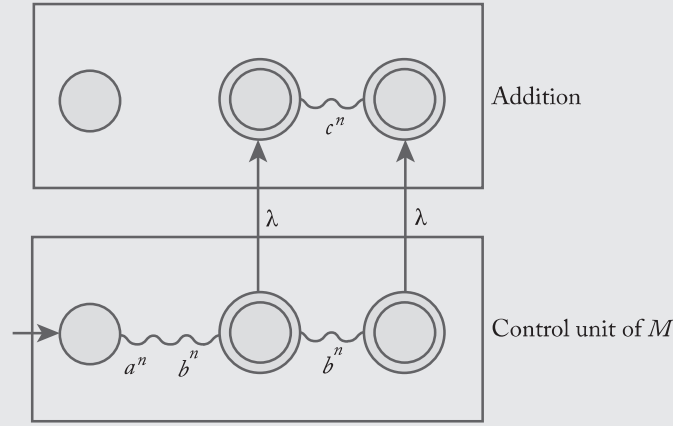


FIGURE 7.4

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ with

$$Q = \{q_0, q_1, \dots, q_n\}.$$

Then consider $\hat{M} = (\hat{Q}, \Sigma, \Gamma, \delta \cup \hat{\delta}, z, \hat{F})$ with

$$\begin{aligned}\hat{Q} &= Q \cup \{\hat{q}_0, \hat{q}_1, \dots, \hat{q}_n\}, \\ \hat{F} &= F \cup \{\hat{q}_i : q_i \in F\},\end{aligned}$$

and $\hat{\delta}$ constructed from δ by including

$$\hat{\delta}(q_f, \lambda, s) = \{(\hat{q}_f, s)\},$$

for all $q_f \in F, s \in \Gamma$, and

$$\hat{\delta}(\hat{q}_i, c, s) = \{(\hat{q}_j, u)\},$$

for all

$$\delta(q_i, b, s) = \{(q_j, u)\},$$

$q_i \in Q, s \in \Gamma, u \in \Gamma^*$. For M to accept $a^n b^n$ we must have

$$(q_0, a^n b^n, z) \vdash_M^* (q_i, \lambda, u),$$

with $q_i \in F$. Because M is deterministic, it must also be true that

$$(q_0, a^n b^{2n}, z) \vdash_M^* (q_i, b^n, u),$$

so that for it to accept $a^n b^{2n}$ we must further have

$$(q_i, b^n, u) \vdash_M^* (q_j, \lambda, u_1),$$

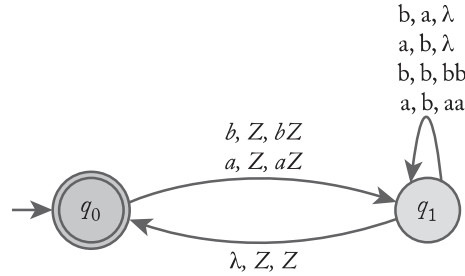
for some $q_j \in F$. But then, by construction

$$(\hat{q}_i, c^n, u) \vdash_{\hat{M}}^* (\hat{q}_j, \lambda, u_1),$$

so that $\hat{M}$ will accept $a^n b^n c^n$. It remains to be shown that no strings other than those in $\hat{L}$ are accepted by $\hat{M}$; this is considered in several exercises at the end of this section. The conclusion is that $\hat{L} = L(\hat{M})$, so that $\hat{L}$ is context-free. But we will show in the next chapter (Example 8.1) that $\hat{L}$ is not context-free. Therefore, our assumption that L is a deterministic context-free language must be false.

INTRODUCTORY EXERCISES

1. In Example 7.10, replace the rule $\delta(q_0, a, 0) = \{(q_1, 10)\}$ with $\delta(q_0, a, 0) = \{(q_1, 110)\}$. Explain why the result is still deterministic.
2. In Example 7.10, add the rule $\delta(q_0, \lambda, 0) = \{(q_2, \lambda)\}$. Show that the new pda is no longer deterministic.
3. The pda below accepts the language $L = \{w : n_b(w) = n_a(w)\}$. What does this tell us about this language?



4. Without actually constructing a pda, discuss why the claim that $L = \{a^n b^k a^n, n, k \geq 1\}$ is a deterministic context-free language is reasonable.

EXERCISES

1. Show that $L = \{a^n b^m, n > m\}$ is a deterministic context-free language.
2. Show that $L = \{a^n b^{2n} : n \geq 0\}$ is a deterministic context-free language.
3. Show that $L = \{a^n b^m, n < m\}$ is a deterministic context-free language.
4. Show that $L = \{a^n b^m : m \geq n + 3\}$ is deterministic.

5. Is the language $L = \{a^n b^n : n \geq 1\} \cup \{b\}$ deterministic?
6. Is the language $L = \{a^n b^n : n \geq 1\} \cup \{a\}$ deterministic?
7. Show that the pushdown automaton in Example 7.4 is not deterministic, but that the language in the example is nevertheless deterministic.
8. For the language L in Exercise 1, show that L^* is a deterministic context-free language.
9. Give reasons one might conjecture that the following language is not deterministic:

$$L = \{a^n b^m c^k : n = m \text{ or } m = k\}.$$

10. Is the language $L = \{a^n b^m : n = m \text{ or } n = m + 1\}$ deterministic?
 11. Is the language $\{wcw^R : w \in \{a, b\}^*\}$ deterministic?
 12. While the language in Exercise 11 is deterministic, the closely related language $L = \{ww^R : w \in \{a, b\}^*\}$ is known to be nondeterministic. Give arguments that make this statement plausible.
 13. Show that $L = \{w \in \{a, b\}^* : n_a(w) \neq n_b(w)\}$ is a deterministic context-free language.
 14. Show that $L = \{a^n b^m, n < 2m\}$ is a deterministic context-free language.
 15. Show that if L_1 is deterministic context-free and L_2 is regular, then the language $L_1 \cup L_2$ is deterministic context-free.
 16. Show that under the conditions of Exercise 15, $L_1 \cap L_2$ is a deterministic context-free language.
 17. Give an example of a deterministic context-free language whose reverse is not deterministic.
-

7.4 GRAMMARS FOR DETERMINISTIC CONTEXT-FREE LANGUAGES*

The importance of deterministic context-free languages lies in the fact that they can be parsed efficiently. We can see this intuitively by viewing the pushdown automaton as a parsing device. Since there is no backtracking involved, we can easily write a computer program for it, and we may expect that it will work efficiently. Since there may be λ -transitions involved, we cannot immediately claim that this will yield a linear-time parser, but it puts us on the right track nevertheless. To pursue this, let us see what grammars might be suitable for the description of deterministic context-free languages. Here we enter a topic important in the study of compilers, but somewhat peripheral to our interests. We will provide only a brief introduction to some important results, referring the reader to books on compilers for a more thorough treatment.

Suppose we are parsing top-down, attempting to find the leftmost derivation of a particular sentence. For the sake of discussion, we use the

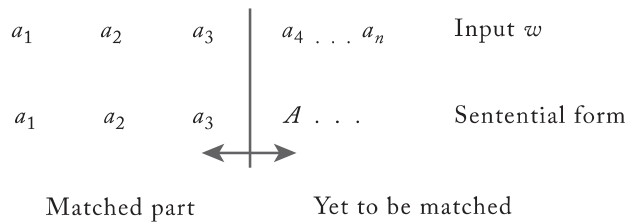


FIGURE 7.5

approach illustrated in Figure 7.5. We scan the input w from left to right, while developing a sentential form whose terminal prefix matches the prefix of w up to the currently scanned symbol. To proceed in matching consecutive symbols, we would like to know exactly which production rule is to be applied at each step. This would avoid backtracking and give us an efficient parser. The question then is whether there are grammars that allow us to do this. For a general context-free grammar, this is not the case, but if the form of the grammar is restricted, we can achieve our goal.

As first case, take the s-grammars introduced in Definition 5.4. From the discussion there, it is clear that at every stage in the parsing we know exactly which production has to be applied. Suppose that $w = w_1w_2$ and that we have developed the sentential form w_1Ax . To get the next symbol of the sentential form matched against the next symbol in w , we simply look at the leftmost symbol of w_2 , say a . If there is no rule $A \rightarrow ay$ in the grammar, the string w does not belong to the language. If there is such a rule, the parsing can proceed. But in this case there is only one such rule, so there is no choice to be made.

Although s-grammars are useful, they are too restrictive to capture all aspects of the syntax of programming languages. We need to generalize the idea so that it becomes more powerful without losing its essential property for parsing. One type of grammar is called an ***LL* grammar**. In an *LL* grammar we still have the property that we can, by looking at a limited part of the input (consisting of the scanned symbol plus a finite number of symbols following it), predict exactly which production rule must be used. The term *LL* is standard usage in books on compilers; the first *L* stands for the fact that the input is scanned from left to right; the second *L* indicates that leftmost derivations are constructed. Every s-grammar is an *LL* grammar, but the concept is more general.

EXAMPLE 7.12

The grammar

$$S \rightarrow aSb|ab$$

is not an s-grammar, but it is an *LL* grammar. In order to determine which production is to be applied, we look at two consecutive symbols

of the input string. If the first is an a and the second a b , we must apply the production $S \rightarrow ab$. Otherwise, the rule $S \rightarrow aSb$ must be used.

We say that a grammar is an $LL(k)$ grammar if we can uniquely identify the correct production, given the currently scanned symbol and a “look-ahead” of the next $k - 1$ symbols. Example 7.12 is an example of an $LL(2)$ grammar.

EXAMPLE 7.13

The grammar

$$S \rightarrow SS | aSb | ab$$

generates the positive closure of the language in Example 7.12. As remarked in Example 5.4, this is the language of properly nested parenthesis structures. The grammar is not an $LL(k)$ grammar for any k .

To see why this is so, look at the derivation of strings of length greater than two. To start, we have available two possible productions $S \rightarrow SS$ and $S \rightarrow aSb$. The scanned symbol does not tell us which is the right one. Suppose we now use a look-ahead and consider the first two symbols, finding that they are aa . Does this allow us to make the right decision? The answer is still no, since what we have seen could be a prefix of a number of strings, including both $aabb$ or $aabbab$. In the first case, we must start with $S \rightarrow aSb$, while in the second it is necessary to use $S \rightarrow SS$. The grammar is therefore not an $LL(2)$ grammar. In a similar fashion, we can see that no matter how many look-ahead symbols we have, there are always some situations that cannot be resolved.

This observation about the grammar does not imply that the language is not deterministic or that no LL grammar for it exists. We can find an LL grammar for the language if we analyze the reason for the failure of the original grammar. The difficulty lies in the fact that we cannot predict how many repetitions of the basic pattern $a^n b^n$ there are until we get to the end of the string, yet the grammar requires an immediate decision. Rewriting the grammar avoids this difficulty. The grammar

$$S \rightarrow aSbS | \lambda$$

is an LL grammar nearly equivalent to the original grammar.

To see this, consider the leftmost derivation of $w = abab$. Then

$$S \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab.$$

We see that we never have any choice. When the input symbol examined is a , we must use $S \rightarrow aSbS$, when the symbol is b or we are at the end of the string, we must use $S \rightarrow \lambda$.

But the problem is not yet completely solved because the new grammar can generate the empty string. We fix this by introducing a new start variable S_0 and a production to ensure that some nonempty string is generated. The final result

$$\begin{aligned} S_0 &\rightarrow aSbS, \\ S &\rightarrow aSbS|\lambda \end{aligned}$$

is then an LL grammar equivalent to the original grammar.

While this informal description of LL grammars is adequate for understanding simple examples, we need a more precise definition if any rigorous results are to be developed. We conclude our discussion with such a definition.

DEFINITION 7.5

Let $G = (V, T, S, P)$ be a context-free grammar. If for every pair of leftmost derivations

$$\begin{aligned} S &\stackrel{*}{\Rightarrow} w_1Ax_1 \Rightarrow w_1y_1x_1 \stackrel{*}{\Rightarrow} w_1w_2, \\ S &\stackrel{*}{\Rightarrow} w_1Ax_2 \Rightarrow w_1y_2x_2 \stackrel{*}{\Rightarrow} w_1w_3, \end{aligned}$$

with $w_1, w_2, w_3 \in T^*$, the equality of the k leftmost symbols of w_2 and w_3 implies $y_1 = y_2$, then G is said to be an $LL(k)$ grammar. (If $|w_2|$ or $|w_3|$ is less than k , then k is replaced by the smaller of these.)

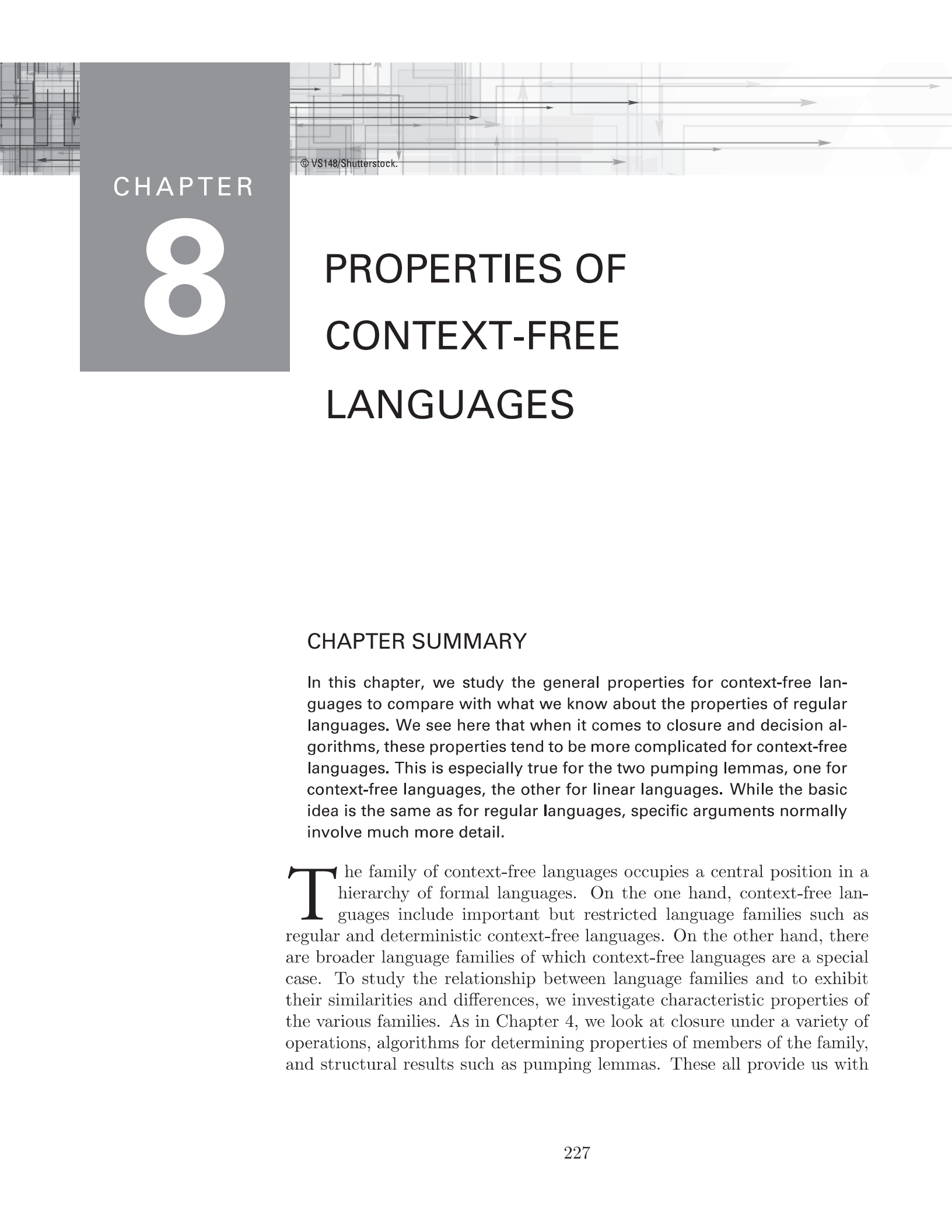
The definition makes precise what has already been indicated. If at any stage in the leftmost derivation (w_1Ax) we know the next k symbols of the input, the next step in the derivation is uniquely determined (as expressed by $y_1 = y_2$).

The topic of LL grammars is an important one in the study of compilers. A number of programming languages can be defined by LL grammars, and many compilers have been written using LL parsers. But LL grammars are not sufficiently general to deal with all deterministic context-free languages. Consequently, there is interest in other, more general deterministic grammars. Particularly important are the so-called LR grammars, which also allow efficient parsing but can be viewed as constructing the derivation

tree from the bottom up. There is a great deal of material on this subject that can be found in books on compilers (e.g., Hunter 1981) or books specifically devoted to parsing methods for formal languages (such as Aho and Ullman 1972). We explore *LL* and *LR* parsers in Chapters 15–17.

EXERCISES

1. Give *LL* grammars for the following languages, assuming $\Sigma = \{a, b, c\}$:
 - (a) $L = \{a^n b^m c^{n+m} : n \geq 1, m \geq 1\}$.
 - (b) $L = \{a^{n+2} b^m c^{n+m} : n \geq 1, m \geq 1\}$.
 - (c) $L = \{a^n b^{n+2} c^m : n \geq 1, m > 2\}$.
 2. Show that the second grammar in Example 7.13 is an *LL* grammar and that it is equivalent to the original grammar.
 3. Show that the grammar for $L = \{w : n_a(w) = n_b(w)\}$ given in Example 1.13 is not an *LL* grammar.
 4. Construct an *LL* grammar for the language $L(aa^*ba) \cup L(abbb^*)$.
 5. Show that any *LL* grammar is unambiguous.
 6. Show that if G is an *LL*(k) grammar, then $L(G)$ is a deterministic context-free language.
 7. Show that a deterministic context-free language is never inherently ambiguous.
 8. Let G be a context-free grammar in Greibach normal form. Describe an algorithm which, for any given k , determines whether or not G is an *LL*(k) grammar.
-



CHAPTER

8

PROPERTIES OF CONTEXT-FREE LANGUAGES

CHAPTER SUMMARY

In this chapter, we study the general properties for context-free languages to compare with what we know about the properties of regular languages. We see here that when it comes to closure and decision algorithms, these properties tend to be more complicated for context-free languages. This is especially true for the two pumping lemmas, one for context-free languages, the other for linear languages. While the basic idea is the same as for regular languages, specific arguments normally involve much more detail.

The family of context-free languages occupies a central position in a hierarchy of formal languages. On the one hand, context-free languages include important but restricted language families such as regular and deterministic context-free languages. On the other hand, there are broader language families of which context-free languages are a special case. To study the relationship between language families and to exhibit their similarities and differences, we investigate characteristic properties of the various families. As in Chapter 4, we look at closure under a variety of operations, algorithms for determining properties of members of the family, and structural results such as pumping lemmas. These all provide us with

a means of understanding relations between the different families as well as for classifying specific languages in an appropriate category.

8.1 TWO PUMPING LEMMAS

The pumping lemma given in Theorem 4.8 is an effective tool for showing that certain languages are not regular. Similar pumping lemmas are known for other language families. Here we will discuss two such results, one for context-free languages in general, the other for a restricted type of context-free language.

The pumping lemma for regular languages is based on the fact that all strings in a regular language must exhibit a certain repetitive pattern. If a language L contains a string that does not follow this pattern, then L cannot be regular. A similar reasoning leads to the pumping lemma for context-free languages. For regular languages, the pattern is a consequence of the finiteness of the representing dfa; with context-free languages, the pattern is most easily seen via the grammar.

Suppose a variable repeats in the sense that $A \xRightarrow{*} vAy$, with $A \xRightarrow{*} x$. The derived sentence will then contain the substring vxy . There will also be sentences in the language that contain substrings x , $vvxyy$, $vvvxyyy$, and so on.

While, in essence, the pumping lemma for context-free languages is no different from the pumping lemma for regular languages, its application is complicated by the fact that now the pumped string consists of two separate parts and that it can occur anywhere in the string.

A Pumping Lemma for Context-Free Languages

THEOREM 8.1

Let L be an infinite context-free language. Then there exists some positive integer m such that any $w \in L$ with $|w| \geq m$ can be decomposed as

$$w = uvxyz, \tag{8.1}$$

with

$$|vxy| \leq m, \tag{8.2}$$

and

$$|vy| \geq 1, \tag{8.3}$$

such that

$$uv^i xy^i z \in L, \tag{8.4}$$

for all $i = 0, 1, 2, \dots$. This is known as the pumping lemma for context-free languages.

Proof: Consider the language $L = \{\lambda\}$, and assume that we have for it a grammar G without unit-productions or λ -productions. Since the length of the string on the right side of any production is bounded, say by k , the length of the derivation of any $w \in L$ must be at least $|w|/k$. Therefore, since L is infinite, there exist arbitrarily long derivations and corresponding derivation trees of arbitrary height.

Consider now such a high derivation tree and some sufficiently long path from the root to a leaf. Since the number of variables in G is finite, there must be some variable that repeats on this path, as shown schematically in Figure 8.1. Corresponding to the derivation tree in Figure 8.1, we have the derivation

$$S \Rightarrow^* uAz \Rightarrow^* uvAyz \Rightarrow^* uvxyz,$$

where u, v, x, y , and z are all strings of terminals. From the above we see that $A \Rightarrow^* vAy$ and $A \Rightarrow^* x$, so all the strings $uv^i xy^i z$, $i = 0, 1, 2, \dots$, can be generated by the grammar and are therefore in L . Furthermore, in the derivations $A \Rightarrow^* vAy$ and $A \Rightarrow^* x$, we can assume that no variable repeats. To see this, look at the sketch of the derivation tree in Figure 8.1. In the subtree T_5 no variable repeats; otherwise, we could just apply the argument

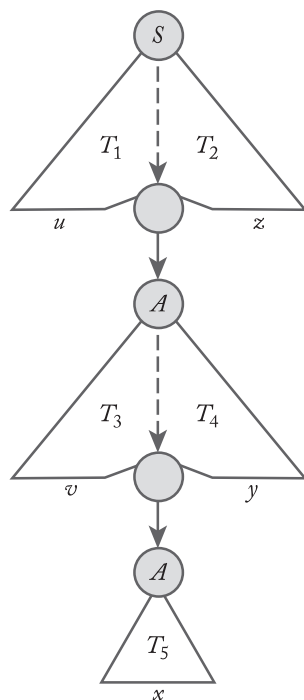


FIGURE 8.1

to this repeating variable. Similarly, we can assume that no variable repeats in the subtrees T_3 and T_4 . Therefore, the lengths of the strings v , x , and y depend only on the productions of the grammar and can be bounded independently of w so that (8.2) holds. Finally, since there are no unit-productions and no λ -productions, v and y cannot both be empty strings, giving (8.3).

This completes the argument that (8.1) to (8.4) hold. ■

This pumping lemma is useful in showing that a language does not belong to the family of context-free languages. Its application is typical of pumping lemmas in general; they are used negatively to show that a given language does not belong to some family. As in Theorem 4.8, the correct argument can be visualized as a game against an intelligent opponent. But now the rules make it a little more difficult for us. For regular languages, the substring xy whose length is bounded by m starts at the left end of w . Therefore, the substring y that can be pumped is within m symbols of the beginning of w . For context-free languages, we only have a bound on $|vxy|$. The substring u that precedes vxy can be arbitrarily long. This gives additional freedom to the adversary, making arguments involving Theorem 8.1 a little more complicated.

EXAMPLE 8.1

Show that the language

$$L = \{a^n b^n c^n : n \geq 0\}$$

is not context-free.

Once the adversary has chosen m , we pick the string $a^m b^m c^m$, which is in L . The adversary now has several choices. If he chooses vxy to contain only a 's, then the pumped string will obviously not be in L . If he chooses a decomposition so that v and y are composed of an equal number of a 's and b 's, then the pumped string $a^k b^k c^m$ with $k \neq m$ can be generated, and again we have generated a string not in L . In fact, the only way the adversary could stop us from winning is to pick vxy so that vy has the same number of a 's, b 's, and c 's. But this is not possible because of restriction (8.2). Therefore, L is not context-free.

If we try the same argument on the language $L = \{a^n b^n\}$, we fail, as we must, since the language is context-free. If we pick any string in L , such as $w = a^m b^m$, the adversary can pick $v = a^k$ and $y = b^k$. Now, no matter what i we choose, the resulting pumped string w_i is in L . Remember, though, that this does not prove that L is context-free; all we can say is that we have been unable to get any conclusion from the

pumping lemma. That L is context-free must come from some other argument, such as the construction of a context-free grammar.

The argument also justifies a claim made in Example 7.11 and allows us to close a gap in that example. The language

$$\widehat{L} = \{a^n b^n\} \cup \{a^n b^{2n}\} \cup \{a^n b^n c^n\}$$

is not context-free. The string $a^m b^m c^m$ is in $\widehat{L}$, but the pumped result is not.

EXAMPLE 8.2

Consider the language

$$L = \{ww : w \in \{a, b\}^*\}.$$

Although this language appears to be very similar to the context-free language of Example 5.1, it is not context-free.

Take the string

$$a^m b^m a^m b^m.$$

There are many ways in which the adversary can now pick vxy , but for all of them we have a winning countermove. For example, for the choice in Figure 8.2, we can use $i = 0$ to get a string of the form

$$a^k b^j a^m b^m, k < m \text{ or } j < m,$$

which is not in L . For other choices by the adversary, similar arguments can be made. We conclude that L is not context-free.

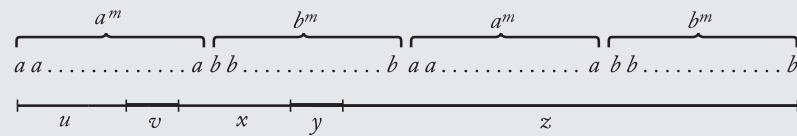


FIGURE 8.2

EXAMPLE 8.3

Show that the language

$$L = \{a^{n!} : n \geq 0\}$$

is not context-free.

Given the opponent's choice for m , we pick $a = a^{m!}$. Obviously, whatever the decomposition is, it must be of the form $v = a^k$, $y = a^l$. Then $w_0 = uxz$ has length $m! - (k + l)$. This string is in L only if

$$m! - (k + l) = j!$$

for some j . But this is impossible, since with $k + l \leq m$,

$$m - (k + l) > (m - 1)!.$$

Therefore, the language is not context-free.

EXAMPLE 8.4

Show that the language

$$L = \{a^n b^j : n = j^2\}$$

is not context-free.

Given m in Theorem 8.1, we pick as our string $a^{m^2} b^m$. The adversary now has several choices. The only one that requires much thought is the one shown in Figure 8.3. Pumping i times will yield a new string with $m^2 + (i - 1)k_1$ a 's and $m + (i - 1)k_2$ b 's. If the adversary takes $k_1 \neq 0$, $k_2 \neq 0$, we can pick $i = 0$. Since

$$\begin{aligned} (m - k_2)^2 &\leq (m - 1)^2 \\ &= m^2 - 2m + 1 \\ &< m^2 - k_1, \end{aligned}$$

the result is not in L . If the opponent picks $k_1 = 0$, $k_2 \neq 0$ or $k_1 \neq 0$, $k_2 = 0$, then again with $i = 0$, the pumped string is not in L . We can conclude from this that L is not a context-free language.

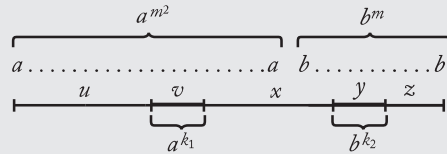


FIGURE 8.3

A Pumping Lemma for Linear Languages

We previously made a distinction between linear and nonlinear context-free grammars. We now make a similar distinction between languages.

DEFINITION 8.1

A context-free language L is said to be linear if there exists a linear context-free grammar G such that $L = L(G)$.

Clearly, every linear language is context-free, but we have not yet established whether or not the converse is true.

EXAMPLE 8.5

The language $L = \{a^n b^n : n \geq 0\}$ is a linear language. A linear grammar for it is given in Example 1.11. The grammar given in Example 1.13 for the language $L = \{w : n_a(w) = n_b(w)\}$ is not linear, so the second language is not necessarily linear.

Of course, just because a specific grammar is not linear does not imply that the language generated by it is not linear. If we want to prove that a language is not linear, we must show that there exists no equivalent linear grammar. We approach this in the usual way, establishing structural properties for linear languages, then showing that some context-free languages do not have a required property.

THEOREM 8.2

Let L be an infinite linear language. Then there exists some positive integer m , such that any $w \in L$, with $|w| \geq m$ can be decomposed as $w = uvxyz$ with

$$|uvyz| \leq m, \quad (8.5)$$

$$|vy| \geq 1, \quad (8.6)$$

such that

$$wv^i xy^i z \in L, \quad (8.7)$$

for all $i = 0, 1, 2, \dots$

Note that the conclusions of this theorem differ from those of Theorem 8.1, since (8.2) is replaced by (8.5). This implies that the strings v and

y to be pumped must now be located within m symbols of the left and right ends of w , respectively. The middle string x can be of arbitrary length.

Proof: Since the language is linear there exists a linear grammar G for it. For the argument it is convenient to assume that G has no unit-productions and no λ -productions. An examination of the proofs of Theorem 6.3 and 6.4 makes it clear that removing unit-productions and λ -productions does not destroy the linearity of the grammar. We can therefore assume that G has the required property.

Consider now the derivation of a string $w \in L(G)$

$$S \xRightarrow{*} uAz \xRightarrow{*} uvAyz \xRightarrow{*} uvxyz = w.$$

Assume, for the moment, that for every $w \in L(G)$, there is a variable A , such that

1. in the partial derivation $S \xRightarrow{*} uAz$ no variable is repeated,
2. in the partial derivation $S \xRightarrow{*} uAz \xRightarrow{*} uvAyz$ no variable except A is repeated,
3. the repetition of A must occur in the first m steps, where m can depend on the grammar, but not on w .

If this is true, then the lengths of u, v, y, z must be bounded independent of w . This in turn implies that (8.5), (8.6), and (8.7) must hold.

To complete the argument, we must still demonstrate that the above conditions hold for every linear grammar. This is not hard to see if we look at sequences in which the variables can occur. We will omit the details here but leave them as an exercise (see Exercise 16 at the end of this section). ■

EXAMPLE 8.6

The language

$$L = \{w : n_a(w) = n_b(w)\}$$

is not linear.

To show this, assume that the language is linear and apply Theorem 8.2 to the string

$$w = a^m b^{2m} a^m.$$

Inequality (8.5) shows that in this case the strings u, v, y, z must all consist entirely of a 's. If we pump this string once, we get $a^{m+k} b^{2m} a^{m+l}$, with either $k \geq 1$ or $l \geq 1$, a result that is not in L . This contradiction of Theorem 8.2 proves that the language is not linear.

This example answers the general question raised on the relation between the families of context-free and linear languages. The family of linear languages is a proper subset of the family of context-free languages.

INTRODUCTORY EXERCISES

1. Consider the following claim: The argument in Example 8.2 is incorrect because the opponent can choose $v = a^k$ from the leftmost string of a 's and $y = a^k$ from the rightmost. Then, no matter what I choose for i , I cannot get a string not in L . So, at least from this analysis, we cannot conclude that L is not context-free. Do you agree?
2. How can the suggested game playing strategy be applied to the analysis of the language $L = \{a^n b^n c^n, n \geq 0\}$? Show the sequence of moves and what the choices and restrictions are at each step. Make sure you consider all opponents' choices and the corresponding winning response.
3. How can the game playing strategy be applied to the analysis of the linear language $L = \{a^n b^k a^l, n \geq 1, l \geq 1, k = n + l\}$. Show the sequence of moves and what the choices and restrictions are at each step. Make sure you consider all opponent's choices and the corresponding winning response. Why would $w = a^{m-2} b^m a a$ be a poor choice in this game?

EXERCISES

1. Show that the language

$$L = \{w : n_a(w) < n_b(w) < n_c(w)\}$$

is not context-free.

2. Show that the language

$$L = \{w \in \{a, b, c\}^* : n_a(w) = n_b(w) \geq n_c(w)\}$$

is not context-free.

3. Determine whether or not the language $L = \{a^n b^i c^j d^k, n + k \leq i + j\}$ is context-free.
4. Show that the language $L = \{a^n : n \text{ is a prime number}\}$ is not context-free.
5. Is the language $L = \{a^n b^m : m = 2^n\}$ context-free?
6. Show that the language $L = \{a^{n^2} : n \geq 0\}$ is not context-free.
7. Show that the following languages on $\Sigma = \{a, b, c\}$ are not context-free:

(a) $L = \{a^n b^j : n \leq j^2\}$.

(b) $L = \{a^n b^j : n \geq (j - 1)^3\}$.

- (c) $L = \{a^n b^j c^k : k = jn\}$.
- (d) $L = \{a^n b^j c^k : k > n, k > j\}$.
- (e) $L = \{a^n b^j c^k : n < j, n \leq k \leq j\}$.
- (f) $L = \{w : n_a(w) = n_b(w) * n_c(w)\}$.
- (g) $L = \{w \in \{a, b, c\}^* : n_a(w) + n_b(w) = 2n_c(w), n_a(w) = n_b(w)\}$.
- (h) $L = \{a^n b^m : n \text{ and } m \text{ are both prime}\}$.
- (i) $L = \{a^n b^m : n \text{ is prime or } m \text{ is prime}\}$.
- (j) $L = \{a^n b^m : n \text{ is prime and } m \text{ is not prime}\}$.

8. Determine whether or not the following languages are context-free:

- (a) $L = \{a^n w w^R b^n : n \geq 0, w \in \{a, b\}^*\}$.
- (b) $L = \{a^n b^j a^n b^j : n \geq 0, j \geq 0\}$.
- (c) $L = \{a^n b^j a^j b^n : n \geq 0, j \geq 0\}$.
- (d) $L = \{a^n b^j a^k b^l : n + j \leq k + l\}$.
- (e) $L = \{a^n b^j a^k b^l : n \leq k, j \leq l\}$.
- (f) $L = \{a^n b^n c^j : n \geq j\}$.
- (g) $L = \{a^n b^n c^k, k = 2n\}$.

9. In Theorem 8.1, find a bound for m in terms of the properties of the grammar G .

10. Determine whether or not the following language is context-free:

$$L = \{w_1 c w_2 : w_1, w_2 \in \{a, b\}^*, w_1 \neq w_2\}.$$

11. Show that the language $L = \{a^n b^n a^m b^m : n \geq 0, m \geq 0\}$ is context-free but not linear.

12. Show that the following language is not linear.

$$L = \{w : n_a(w) \leq n_b(w)\}.$$

13. Show that the language $L = \{w \in \{a, b, c\}^* : n_a(w) + n_b(w) = n_c(w)\}$ is context-free, but not linear.

14. Determine whether or not the language $L = \{a^n b^j : j \leq n \leq 2j - 1\}$ is linear.

15. Determine whether or not the language $L = \{a^n b^m c^n\} \cup \{a^n b^n c^m\}$ is linear.

16. Let G be a linear grammar with k variables. Show that when we write any sequence of variables, there must be some variable A that repeats so that

- (a) the first occurrence of A must be in position $p \leq k$,
- (b) the repetition of A must be no later than $q \leq k + 1$, and
- (c) there can be no other repeating variable between positions p and q .

17. Justify the claim made in Theorem 8.2 that for any linear language (not containing λ) there exists a linear grammar without λ -productions and unit-productions.
18. Consider the set of all strings a/b , where a and b are positive decimal integers such that $a < b$. The set of strings then represents all possible decimal fractions. Determine whether or not this is a context-free language.
19. Show that the complement of the language in Exercise 7 is not context-free.
20. Is the language $L = \{a^{nm} : n \text{ and } m \text{ are prime numbers}\}$ context-free?
21. It is known that the language

$$L = \{a^n b^n c^m : n \neq m\}$$

is not context-free. (See the next exercise.) Show that, in spite of this, it is not possible to use Theorem 8.1 to prove it.

22. **Ogden's lemma** is an extension of Theorem 8.1 that necessitates some changes in the way the pumping lemma game is played. In particular,
 - (a) You can choose any $w \in L$ with $|w| \geq m$, but you must mark at least m symbols in w . You can choose which symbols to mark.
 - (b) The opponent must select the decomposition $w = uvxyz$ with the additional restriction that either vx or xy must have at least one marked position.

Note that Theorem 8.1 is a special case of Ogden's lemma in which all symbols of w are marked. Show how Ogden's lemma can be used to prove that the language in the previous exercise is not context-free, and conclude from this that Ogden's lemma is more powerful than Theorem 8.1.

8.2 CLOSURE PROPERTIES AND DECISION ALGORITHMS FOR CONTEXT-FREE LANGUAGES

In Chapter 4 we looked at closure under certain operations and algorithms to decide on the properties of the family of regular languages. On the whole, the questions raised there had easy answers. When we ask the same questions about context-free languages, we encounter more difficulties. First, closure properties that hold for regular languages do not always hold for context-free languages. When they do, the arguments needed to prove them are often quite complicated. Second, many intuitively simple and important questions about context-free languages cannot be answered. This statement may seem at first surprising and will need to be elaborated as we proceed. In this section, we provide only a sample of some of the most important results.

Closure of Context-Free Languages

THEOREM 8.3

The family of context-free languages is closed under union, concatenation, and star-closure.

Proof: Let L_1 and L_2 be two context-free languages generated by the context-free grammars $G_1 = (V_1, T_1, S_1, P_1)$ and $G_2 = (V_2, T_2, S_2, P_2)$, respectively. We can assume without loss of generality that the sets V_1 and V_2 are disjoint.

Consider now the language $L(G_3)$, generated by the grammar

$$G_3 = (V_1 \cup V_2 \cup \{S_3\}, T_1 \cup T_2, S_3, P_3),$$

where S_3 is a variable not in $V_1 \cup V_2$. The productions of G_3 are all the productions of G_1 and G_2 , together with an alternative starting production that allows us to use one or the other grammars. More precisely,

$$P_3 = P_1 \cup P_2 \cup \{S_3 \rightarrow S_1 | S_2\}.$$

Obviously, G_3 is a context-free grammar, so that $L(G_3)$ is a context-free language. But it is easy to see that

$$L(G_3) = L_1 \cup L_2. \quad (8.8)$$

Suppose, for instance, that $w \in L_1$. Then

$$S_3 \Rightarrow S_1 \xRightarrow{*} w$$

is a possible derivation in grammar G_3 . A similar argument can be made for $w \in L_2$. Also, if $w \in L(G_3)$, then either

$$S_3 \Rightarrow S_1 \quad (8.9)$$

or

$$S_3 \Rightarrow S_2 \quad (8.10)$$

must be the first step of the derivation. Suppose (8.9) is used. Since sentential forms derived from S_1 have variables in V_1 , and V_1 and V_2 are disjoint, the derivation

$$S_1 \xRightarrow{*} w$$

can involve productions in P_1 only. Hence w must be in L_1 . Alternatively, if (8.10) is used first, then w must be in L_2 and it follows that $L(G_3)$ is the union of L_1 and L_2 .

Next, consider

$$G_4 = (V_1 \cup V_2 \cup \{S_4\}, T_1 \cup T_2, S_4, P_4).$$

Here again S_4 is a new variable and

$$P_4 = P_1 \cup P_2 \cup \{S_4 \rightarrow S_1 S_2\}.$$

Then

$$L(G_4) = L(G_1) L(G_2)$$

follows easily.

Finally, consider $L(G_5)$ with

$$G_5 = (V_1 \cup \{S_5\}, T_1, S_5, P_5),$$

where S_5 is a new variable and

$$P_5 = P_1 \cup \{S_5 \rightarrow S_1 S_5 | \lambda\}.$$

Then

$$L(G_5) = L(G_1)^*.$$

Thus we have shown that the family of context-free languages is closed under union, concatenation, and star-closure. ■

THEOREM 8.4

The family of context-free languages is not closed under intersection and complementation.

Proof: Consider the two languages

$$L_1 = \{a^n b^n c^m : n \geq 0, m \geq 0\}$$

and

$$L_2 = \{a^n b^m c^m : n \geq 0, m \geq 0\}.$$

There are several ways one can show that L_1 and L_2 are context-free. For instance, a grammar for L_1 is

$$\begin{aligned} S &\rightarrow S_1 S_2, \\ S_1 &\rightarrow a S_1 b | \lambda, \\ S_2 &\rightarrow c S_2 | \lambda. \end{aligned}$$

Alternatively, we note that L_1 is the concatenation of two context-free languages, so it is context-free by Theorem 8.3. But

$$L_1 \cap L_2 = \{a^n b^n c^n : n \geq 0\},$$

which we have already shown not to be context-free. Thus, the family of context-free languages is not closed under intersection.

The second part of the theorem follows from Theorem 8.3 and the set identity

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}.$$

If the family of context-free languages were closed under complementation, then the right side of the above expression would be a context-free language for any context-free L_1 and L_2 . But this contradicts what we have just shown, that the intersection of two context-free languages is not necessarily context-free. Consequently, the family of context-free languages is not closed under complementation. ■

While the intersection of two context-free languages may produce a language that is not context-free, the closure property holds if one of the languages is regular.

THEOREM 8.5

Let L_1 be a context-free language and L_2 be a regular language. Then $L_1 \cap L_2$ is context-free.

Proof: Let $M_1 = (Q, \Sigma, \Gamma, \delta_1, q_0, z, F_1)$ be an npda that accepts L_1 and $M_2 = (P, \Sigma, \delta_2, p_0, F_2)$ be a dfa that accepts L_2 . We construct a push-down automaton $\hat{M} = (\hat{Q}, \Sigma, \Gamma, \hat{\delta}, \hat{q}_0, z, \hat{F})$ that simulates the parallel action of M_1 and M_2 : Whenever a symbol is read from the input string, $\hat{M}$ simultaneously executes the moves of M_1 and M_2 . To this end we let

$$\begin{aligned}\hat{Q} &= Q \times P, \\ \hat{q}_0 &= (q_0, p_0), \\ \hat{F} &= F_1 \times F_2,\end{aligned}$$

and define $\hat{\delta}$ such that

$$((q_k, p_l), x) \in \hat{\delta}((q_i, p_j), a, b)$$

if and only if

$$(q_k, x) \in \delta_1(q_i, a, b)$$

and

$$\delta_2(p_j, a) = p_l.$$

In this, we also require that if $a = \lambda$, then $p_j = p_l$. In other words, the states of $\hat{M}$ are labeled with pairs (q_i, p_j) , representing the respective states

in which M_1 and M_2 can be after reading a certain input string. It is a straightforward induction argument to show that

$$((q_0, p_0), w, z) \vdash_{\hat{M}}^* ((q_r, p_s), \lambda, x),$$

with $q_r \in F_1$ and $p_s \in F_2$ if and only if

$$(q_0, w, z) \vdash_{M_1}^* (q_r, \lambda, x),$$

and

$$\delta^*(p_0, w) = p_s.$$

Therefore, a string is accepted by $\hat{M}$ if and only if it is accepted by M_1 and M_2 , that is, if it is in $L(M_1) \cap L(M_2) = L_1 \cap L_2$. ■

The property addressed by this theorem is called closure under **regular intersection**. Because of the result of the theorem, we say that the family of context-free languages is closed under regular intersection. This closure property is sometimes useful for simplifying arguments in connection with specific languages.

EXAMPLE 8.7

Show that the language

$$L = \{a^n b^n : n \geq 0, n \neq 100\}$$

is context-free.

It is possible to prove this claim by constructing a pda or a context-free grammar for the language, but the process is tedious. We can get a much neater argument with Theorem 8.5.

Let

$$L_1 = \{a^{100} b^{100}\}.$$

Then, because L_1 is finite, it is regular. Also, it is easy to see that

$$L = \{a^n b^n : n \geq 0\} \cap \bar{L}_1.$$

Therefore, by the closure of regular languages under complementation and the closure of context-free languages under regular intersection, the desired result follows.

EXAMPLE 8.8

Show that the language

$$L = \{w \in \{a, b, c\}^* : n_a(w) = n_b(w) = n_c(w)\}$$

is not context-free.

The pumping lemma can be used for this, but again we can get a much shorter argument using closure under regular intersection. Suppose that L were context-free. Then

$$L \cap L(a^*b^*c^*) = \{a^n b^n c^n : n \geq 0\}$$

would also be context-free. But we already know that this is not so. We conclude that L is not context-free.

Closure properties of languages play an important role in the theory of formal languages and many more closure properties for context-free languages can be established. Some additional results are explored in the exercises at the end of this section.

Some Decidable Properties of Context-Free Languages

By putting together Theorems 5.2 and 6.6, we have already established the existence of a membership algorithm for context-free languages. This is of course an essential feature of any language family useful in practice. Other simple properties of context-free languages can also be determined. For the purpose of this discussion, we assume that the language is described by its grammar.

THEOREM 8.6

Given a context-free grammar $G = (V, T, S, P)$, there exists an algorithm for deciding whether or not $L(G)$ is empty.

Proof: For simplicity, assume that $\lambda \notin L(G)$. Slight changes have to be made in the argument if this is not so. We use the algorithm for removing useless symbols and productions. If S is found to be useless, then $L(G)$ is empty; if not, then $L(G)$ contains at least one element. ■

THEOREM 8.7

Given a context-free grammar $G = (V, T, S, P)$, there exists an algorithm for determining whether or not $L(G)$ is infinite.

Proof: We assume that G contains no λ -productions, no unit-productions, and no useless symbols. Suppose the grammar has a repeating variable in the sense that there exists some $A \in V$ for which there is a derivation

$$A \Rightarrow^* xAy.$$

Since G is assumed to have no λ -productions and no unit-productions, x and y cannot be simultaneously empty. Since A is neither nullable nor a useless symbol, we have

$$S \Rightarrow^* uAv \Rightarrow^* w$$

and

$$A \Rightarrow^* z,$$

where u , v , and z are in T^* . But then

$$S \Rightarrow^* uAv \Rightarrow^* ux^n Ay^n v \Rightarrow^* ux^n zy^n v$$

is possible for all n , so that $L(G)$ is infinite.

If no variable can ever repeat, then the length of any derivation is bounded by $|V|$. In that case, $L(G)$ is finite.

Thus, to get an algorithm for determining whether $L(G)$ is finite, we need only to determine whether the grammar has some repeating variables. This can be done simply by drawing a dependency graph for the variables in such a way that there is an edge (A, B) whenever there is a corresponding production

$$A \rightarrow xBy.$$

Then any variable that is at the base of a cycle is a repeating one. Consequently, the grammar has a repeating variable if and only if the dependency graph has a cycle.

Since we now have an algorithm for deciding whether a grammar has a repeating variable, we have an algorithm for determining whether or not $L(G)$ is infinite. ■

Somewhat surprisingly, other simple properties of context-free languages are not so easily dealt with. As in Theorem 4.7, we might look for an algorithm to determine whether two context-free grammars generate the same language. But it turns out that there is no such algorithm. For the moment, we do not have the technical machinery for properly defining the meaning of “there is no algorithm,” but its intuitive meaning is clear. This is an important point to which we will return later.